

Edital:	Edital PIIC 2025/2026
Área do Conhecimento (CNPq):	Ciência da Computação
Subárea do Conhecimento (CNPq):	Metodologia e Técnicas da Computação
Título do Projeto:	Pesquisa Operacional e Inovação em Logística Regional
Título do Subprojeto:	Otimização Dinâmica de Rotas para Coleta de Resíduos Sólidos: Adaptação do Algoritmo de Solomon com Integração IoT
Orientador(a):	Maria Claudia Silva Boeres
Estudante:	Emilly Lopes Das Neves

Estudante apresenta alguma necessidade de acessibilidade?	Sim ()	Não (X)
Se sim, informar o recurso ou suporte de acessibilidade que necessita.		

Comparação experimental de cinco heurísticas clássicas para o VRPTW, com aplicação à coleta de resíduos sólidos via Digital Twin

Resumo. Este trabalho apresenta uma comparação experimental, sob condições controladas e reproduzíveis, de cinco heurísticas e meta-heurísticas clássicas para o Problema de Roteamento de Veículos com Janelas de Tempo (VRPTW): a heurística de inserção I1 de Solomon (1987), a Descida em Vizinhança Variável (VND) (Hansen e Mladenović, 2001), o GRASP (Feo e Resende, 1995) nas formas fixa e reativa (Prais e Ribeiro, 2000), e a Busca Tabu (Glover, 1989). Todos os métodos compartilham o mesmo núcleo (leitor de instâncias, função de distância, avaliador de viabilidade, conjunto de vizinhanças e gerador pseudoaleatório), de modo que as diferenças observadas decorram exclusivamente da estratégia de busca. A função-objetivo otimizada é a distância total sob a convenção do 12º Desafio DIMACS; a contagem de veículos é reportada como medida secundária. Avaliamos as 56 instâncias de 100 clientes de Solomon, com calibração de parâmetros por *irace* (López-Ibáñez et al., 2016) sob

divisão treino/teste, parada por número de iterações sem melhora, e comparação estatística por Friedman seguida do pós-teste de Nemenyi (Demšar, 2006). A viabilidade (cobertura de todos os clientes, capacidade e janelas de tempo) é garantida por construção e *reverificada de forma independente* em todas as soluções. Por fim, um *Digital Twin* aplica os algoritmos calibrados a um cenário de coleta de resíduos com localizações reais de Vitória/ES, como prova de conceito.

Palavras-chave: VRPTW; meta-heurísticas; GRASP; Busca Tabu; comparação experimental; coleta de resíduos.

1 Introdução

A gestão de resíduos sólidos urbanos impõe desafios operacionais, econômicos e ambientais aos municípios. Sistemas convencionais de coleta, baseados em rotas fixas e frequências predeterminadas, geram deslocamentos desnecessários e respondem mal a situações de enchimento crítico dos recipientes (Pardini et al., 2020; Lozano et al., 2018). A literatura aponta que integrar monitoramento em tempo real e otimização de rotas pode tornar a coleta mais eficiente (Lozano et al., 2018; Barth et al., 2023).

Este subprojeto de Iniciação Científica combina sensoriamento IoT via LoRaWAN (Ramson et al., 2022; Almeida e Borin, 2021) com heurísticas baseadas no algoritmo de Solomon (1987) para o VRPTW, além de um *Digital Twin* (gêmeo digital, definido na Seção 5) para simulação e visualização. A pergunta de pesquisa é: *de que modo a comparação criteriosa de heurísticas clássicas para o VRPTW pode fundamentar um planejamento de coleta mais responsivo e eficiente, preservando a factibilidade temporal das rotas?*

A contribuição deste relatório é metodológica e experimental, organizada em três camadas: (i) um **benchmark estático** sobre as instâncias clássicas de Solomon, que estabelece a base de comparação justa e calibra os algoritmos; (ii) uma **análise estatística** do desempenho relativo dos cinco métodos; e (iii) uma **aplicação dinâmica** (Digital Twin) que transfere os algoritmos calibrados a um cenário de coleta de resíduos. O *benchmark* estático é a base metodológica sobre a qual a aplicação dinâmica se apoia: os mesmos operadores e parâmetros validados nas instâncias de Solomon são reaproveitados no Digital Twin.

2 O Problema de Roteamento com Janelas de Tempo

2.1 Formulação

A formulação a seguir é apresentada em sua forma abstrata — clientes genéricos, um depósito, uma frota —, porque é nessa forma que o problema é resolvido no *benchmark* de Solomon (Seção 4). No cenário de coleta da Seção 5, esses papéis ganharão nomes concretos: os clientes serão os pontos de entrega voluntária de recicláveis (PEVs) de Vitória/ES, o depósito será o galpão da associação de triagem (AMARIV), a demanda q_i refletirá o enchimento de cada lixeira e a janela de tempo, o prazo para coletá-la antes do transbordo.

Seja um grafo completo $G = (V, A)$, com $V = \{0, 1, \dots, n\}$, em que o vértice 0 é o depósito e $C = \{1, \dots, n\}$ são os clientes. A cada cliente i associam-se: demanda $q_i \geq 0$, janela de tempo $[e_i, \ell_i]$ e tempo de serviço s_i . O depósito tem janela $[e_0, \ell_0]$ (o horizonte) e $q_0 = s_0 = 0$. Uma frota homogênea de veículos de capacidade Q parte e retorna ao depósito. A cada arco (i, j) associam-se uma distância d_{ij} e um tempo de viagem t_{ij} .

Uma *rota* é uma sequência $r = \langle 0, c_1, \dots, c_m, 0 \rangle$. Definindo o instante de chegada $a_{c_1} = t_{0,c_1}$ e, para $k > 1$, o início de serviço $b_{c_k} = \max(a_{c_k}, e_{c_k})$ com $a_{c_k} = b_{c_{k-1}} + s_{c_{k-1}} + t_{c_{k-1},c_k}$, a rota é *viável* se

$$b_{c_k} \leq \ell_{c_k} \quad \forall k, \quad \sum_k q_{c_k} \leq Q, \quad b_{c_m} + s_{c_m} + t_{c_m,0} \leq \ell_0. \quad (1)$$

Uma *solução* S é um conjunto de rotas $\mathcal{R}(S)$ em que cada cliente é visitado **exatamente uma vez**. Suas duas grandezas de interesse são a **distância total** e o **número de veículos**:

$$D(S) = \sum_{r \in \mathcal{R}(S)} \sum_{(i,j) \in r} d_{ij}, \quad K(S) = |\mathcal{R}(S)|. \quad (2)$$

O objetivo primário deste trabalho é minimizar a distância total $D(S)$, conforme detalhado na Seção 2.2.

2.2 Convenção de distância e função-objetivo

Adotamos a convenção do 12º Desafio DIMACS para o VRPTW (DIMACS, 2022): a distância de cada arco é a euclidiana **truncada** a uma casa decimal, $d_{ij} = \lfloor 10 e_{ij} \rfloor / 10$ (com e_{ij} a distância euclidiana), e o tempo de viagem é igual à distância ($t_{ij} = d_{ij}$). O truncamento é a padronização adotada pelo desafio para tornar custos comparáveis

entre implementações; os resultados originais de Solomon (1987) foram computados em precisão total — distinção relevante ao confrontar valores publicados sob convenções diferentes, discutida na Seção 4.7. As regras do desafio adotam explicitamente a minimização *apenas da distância total* (não a hierarquia veículos→distância): verifica-se que a solução respeita o número máximo de veículos disponíveis, mas não há bônus por usar menos. Essa convenção foi escolhida por ser o padrão internacional moderno do desafio e por ser *verificável*: ela reproduz exatamente os custos das soluções de referência (Seção 4.1). Formalmente, o objetivo otimizado pela busca é

$$\min_{S \text{ viável}} D(S), \quad (3)$$

com a distância relatada a uma casa decimal e o número de veículos $K(S)$ como medida secundária. Essa formulação difere da hierarquia *lexicográfica* clássica de Solomon (1987),

$$\text{lex min}_{S \text{ viável}} (K(S), D(S)), \quad (4)$$

que minimiza *primeiro* o número de veículos e, entre soluções de mesma frota, a distância. Sob a Equação (3), a busca não minimiza o número de veículos diretamente; esse número é uma consequência da construção e só diminui quando um movimento entre rotas esvazia completamente uma rota. A Seção 4.7 discute as implicações dessa escolha sobre a comparação com os melhores valores conhecidos.

2.3 Invariantes de viabilidade

Um requisito central deste trabalho é que **toda** solução produzida — pela construção e por todo algoritmo de melhoria — satisfaça a Equação (1) e a cobertura de clientes. Garantimos isso por três mecanismos redundantes:

1. **Avaliação por rota.** Cada rota mantém os vetores de chegada, início e partida, recalculados a cada modificação; a viabilidade da janela e da capacidade é verificada nessa passagem.
2. **Critério Push Forward.** Para inserções, calcula-se a *folga (slack)* máxima de deslocamento para frente de cada cliente sem violar a própria janela nem as subsequentes; uma inserção só é aceita se o atraso que provoca couber nessa folga. Isso permite testar viabilidade em $O(1)$ por posição candidata (Solomon, 1987).
3. **Verificador independente.** Toda solução final é reavaliada do zero por um verificador que percorre cada rota e checa as quatro restrições — cobertura ($\forall i \in C$ visitado uma vez), capacidade, janelas e retorno ao depósito —, além do respeito

à frota máxima declarada na instância, sem reaproveitar nenhuma estrutura da busca. Funciona como um *portão* de corretude. Reavaliar do zero importa por uma razão de teste de software: a busca opera sobre caches incrementais (os vetores de instantes e folgas da Seção 3.2), e um defeito nesses caches produziria soluções que *parecem* viáveis à própria busca. O verificador recalcula tudo apenas a partir da sequência de clientes, da instância e da matriz de distâncias; um erro de estado interno da busca, por isso, não tem como se autovalidar.

A consequência prática (Seção 4.2) é que, em todas as execuções realizadas neste trabalho, **nenhuma** solução inviável foi produzida por qualquer algoritmo — e o pipeline de experimentos trata uma eventual violação como erro fatal, interrompendo o estudo em vez de registrá-la e seguir. O critério de aceitação de *todos* os operadores de melhoria descarta movimentos inviáveis *antes* de aplicá-los (Seção 3.9); a viabilidade é, portanto, um invariante preservado a cada passo.

3 Metodologia

Todos os algoritmos foram implementados em C++20 sobre um núcleo único: um leitor das instâncias de Solomon, a matriz de distâncias truncadas, o avaliador de viabilidade, o conjunto de vizinhanças e um gerador pseudoaleatório com semente registrada. Compartilhar esse núcleo é o que torna a comparação **justa**: com leitor, distância, vizinhanças e geração de números aleatórios idênticos, a estratégia de busca passa a ser a única origem possível das diferenças de desempenho. Daqui em diante, quando o contexto for claro, abreviamos GRASP Reativo por *Reativo* e Busca Tabu por *Tabu*.

3.1 Arquitetura do solver e fluxo de execução

O sistema é organizado em um *núcleo compartilhado* e módulos de algoritmo. O núcleo contém: (i) o **leitor de instâncias**, que interpreta o formato de Solomon (coordenadas, demanda, janela, serviço, capacidade); (ii) a **matriz de distâncias**, único ponto onde as distâncias são calculadas, garantindo valores idênticos a todos os métodos — e que admite, no Digital Twin, matrizes *explícitas* de distância e tempo (Seção 5.2); (iii) a estrutura de **rota**, que mantém em cache os instantes de chegada, início e partida e a *folga* Push Forward de cada cliente, recalculados a cada modificação; (iv) o **avaliador**, que computa a distância total (objetivo) e a chave de comparação; (v) o **conjunto de vizinhanças**, com a avaliação de viabilidade de cada movimento candidato; (vi) o **gerador pseudoaleatório**, semeado de forma determinística; (vii) o **critério de parada**; e (viii) o **verificador independente**.

O fluxo de uma execução é o seguinte: (1) o arquivo da instância é lido e (2) a matriz de distâncias é construída; (3) gera-se uma solução inicial — a I1 (Algoritmo 1) para VND e Busca Tabu, ou a construção gulosa-aleatorizada para o GRASP; (4) a busca local/meta-heurística refina a solução, consultando o avaliador e as vizinhanças e aceitando movimentos conforme os critérios de aceitação (Seção 3.9), até o critério de parada (Seção 3.12); (5) a solução final é **revalidada** do zero pelo verificador independente; e (6) os artefatos são gravados: o arquivo `.sol` (rotas), uma linha de CSV (distância, veículos, iterações, tempo, viabilidade), o traço de convergência e, opcionalmente, instantâneos por movimento.

Em torno do solver, um *pipeline* em Python executa, para cada trinca (algoritmo, instância, semente), o binário do solver; agrega as métricas; aplica os testes estatísticos; e gera as figuras. A calibração por *irace* (Seção 3.13) e a re-execução completa do estudo são orquestradas por *scripts* dedicados, de modo que todo o experimento é reproduzível por um único comando. Por fim, o **Digital Twin** (Seção 5) reaproveita o mesmo binário: a cada ciclo, os sensores atualizam o enchimento das lixeiras, as lixeiras críticas tornam-se uma instância VRPTW, o solver (re)planeja as rotas com os parâmetros calibrados, e os resultados são visualizados.

3.2 Detalhes de implementação

Linguagem e construção. O solver é escrito em C++20 e compilado com `g++ -O2`, *sem dependências externas* (sem CMake nem bibliotecas de terceiros); a construção e a suíte de 34 testes unitários são geridas por um `Makefile`. O ferramental experimental é em Python (biblioteca padrão + `numpy`, `pandas`, `matplotlib`, `scipy`, `scikit-posthocs`).

Estruturas de dados. A `Instance` guarda, para cada nó, as coordenadas (x, y) , a demanda, a janela $[e_i, \ell_i]$ e o serviço, além da capacidade Q . A `DistanceMatrix` é um vetor plano $n \times n$, em ordem de linha (*row-major*), preenchido uma única vez com $\lfloor 10 e_{ij} \rfloor / 10$. Cada `Route` armazena a sequência de clientes (o depósito é implícito nas pontas) e *vetores em cache* dos instantes de chegada, início e partida e da folga `Push Forward`, recalculados por `recompute()` em uma passada para frente (tempos) e uma para trás (folgas); guarda também a carga e a distância acumuladas.

Avaliação dos movimentos. Cada operador de vizinhança monta a(s) sequência(s) candidata(s) e as avalia por `evaluate_seq` — uma simulação completa da rota, em tempo proporcional ao seu comprimento, que devolve viabilidade, distância e carga; apenas movimentos *viáveis* (e melhorantes, ou admissíveis na Busca Tabu) são retidos por um rastreador do melhor movimento. As inserções da construção são testadas

em $O(1)$ pela folga Push Forward (`eval_insert`). A Busca Tabu mantém um vetor `tabu_until` indexado por cliente: um movimento é tabu se algum dos seus clientes-atributo (o primeiro cliente de cada segmento movido) foi movido há menos de *tenure* iterações (a duração da proibição), com aspiração pelo melhor global; os conceitos do método são definidos na Seção 3.8. O critério de parada usa um relógio monotônico apenas como teto de segurança; o contador de iterações sem melhora é o critério primário.

Reprodutibilidade e instrumentação. O gerador pseudoaleatório de cada execução é semeado combinando a semente da execução (inteiro de 1 a R , sendo R o número de execuções por instância — 30 neste estudo —, registrado no CSV) com um *hash* FNV-1a do nome da instância, que descorrelaciona os fluxos aleatórios entre instâncias que compartilham a mesma semente. O FNV-1a é especificado byte a byte, de modo que o mesmo par (instância, semente) produz o mesmo fluxo aleatório em qualquer plataforma — a reprodutibilidade não depende do compilador. A cada execução o solver grava: o arquivo `.sol` (rotas e custo), uma linha de CSV (algoritmo, instância, semente, distância, veículos, tempo, viabilidade e iterações), o traço de convergência (tempo, melhor custo) e, opcionalmente, instantâneos por movimento para as figuras de evolução. O *Validator*, independente, reavalia a solução do zero — o *portão* de correteude.

Pipeline experimental. O `runner.py` invoca o binário do solver via `subprocess` para cada (algoritmo, instância, semente), em paralelo (`ThreadPoolExecutor`), consolidando o CSV mestre e aplicando os parâmetros calibrados (`tuned.json`). A calibração usa o *irace* por meio de um *target-runner* que imprime o *gap* — a distância percentual entre a solução obtida e a melhor solução conhecida da instância, métrica definida formalmente na Seção 4; os módulos de análise computam as agregações, os testes de Friedman/Nemenyi (`scipy` + `scikit-posthocs`) e as figuras.

Como o C++ e o Python conversam. Os dois lados não compartilham memória nem se ligam por biblioteca: toda a comunicação é por arquivos e pela linha de comando. Na ida, o Python entrega o problema como texto — o arquivo da instância no formato de Solomon e, no cenário viário, as matrizes explícitas de distância e tempo, também arquivos-texto apontados pelas opções `--dist-matrix/--time-matrix` — e as escolhas da execução (algoritmo, semente, critério de parada, parâmetros calibrados) como argumentos do binário. Na volta, o solver responde com os artefatos já listados no fluxo de execução (Seção 3.1) — o `.sol`, a linha de CSV e o traço de convergência —, que o *pandas* agrega no CSV mestre de onde saem todas as tabelas e figuras. A fronteira estreita tem duas vantagens práticas. Cada artefato intermediário pode ser lido e au-

ditado fora do programa que o gerou: o verificador, por exemplo, relê qualquer `.sol` e refaz a checagem de viabilidade sem tocar no código da busca. E o mesmo contrato serve ao *benchmark* e ao Digital Twin, que escreve a instância de cada ciclo e lê de volta as rotas do mesmo modo.

Digital Twin. Os sensores geram o enchimento por um processo de Poisson não homogêneo (`numpy`); a cada ciclo, as lixeiras críticas são escritas como uma instância e roteadas pelo mesmo binário. Para a malha viária real, `road_network.py` obtém as matrizes de distância e tempo pelo serviço *table* do OSRM e a geometria das ruas pela Overpass (cacheadas em JSON), e o solver as recebe pelas opções `--dist-matrix` e `--time-matrix` (Seção 5.2). Esse desenho mantém o *benchmark* euclidiano inalterado: na ausência de matriz de tempo, o tempo de viagem é igual à distância, e a suíte de testes permanece idêntica.

3.3 Construção: heurística de inserção I1 de Solomon

A construção segue a heurística sequencial I1 de Solomon (1987). A I1 não minimiza a Equação (3) diretamente: seu papel é produzir, de forma gulosa e determinística, a solução viável de partida sobre a qual todos os métodos de melhoria operam. Os critérios locais c_1 e c_2 , definidos a seguir, controlam o acréscimo de distância de cada inserção, o que alinha a construção ao objetivo de distância, mas sem garantia de otimalidade. Cada rota é iniciada (*semente*) com o cliente não roteado mais distante do depósito. Em seguida, repetidamente, para cada cliente não roteado u determina-se a melhor posição de inserção segundo o critério de custo c_1 e escolhe-se o cliente que maximiza o critério c_2 . Insere-se o melhor cliente enquanto houver inserção viável; quando não houver, abre-se uma nova rota. Para uma inserção de u entre vizinhos (i, j) de uma rota,

$$c_{11}(i, u, j) = d_{iu} + d_{uj} - \mu d_{ij}, \quad c_{12}(i, u, j) = b_{ju} - b_j, \quad (5)$$

$$c_1(i, u, j) = \alpha_1 c_{11}(i, u, j) + \alpha_2 c_{12}(i, u, j), \quad c_2(u) = \lambda d_{0u} - c_1^{\min}(u), \quad (6)$$

em que c_{11} é o acréscimo de *distância* da inserção e c_{12} é o *deslocamento temporal* (push-forward) que u provoca no sucessor — b_j é o início de serviço em j antes da inserção e b_{ju} o início após inserir u . A quantidade $c_1^{\min}(u) = \min_{(i,j)} c_1(i, u, j)$ é o custo da *melhor* inserção de u na rota corrente, com o mínimo tomado apenas sobre as posições (i, j) *viáveis* — as que respeitam a capacidade e a folga Push Forward (Seção 2.3); se nenhuma posição é viável, $c_1^{\min}(u)$ e $c_2(u)$ ficam indefinidos e u não concorre nesta rodada. Usamos a configuração canônica $\mu = 1$, $\lambda = 2$, $\alpha_1 = 1$, $\alpha_2 = 0$, fixa para

todos os métodos (não calibrada). Com $\alpha_2 = 0$ emprega-se a variante *baseada em distância* de Solomon: o termo temporal c_{12} é anulado pelo peso $\alpha_2 = 0$, de modo que $c_1(i, u, j) = c_{11}(i, u, j)$ — coerente com o objetivo DIMACS; μ pondera a economia de remover o arco (i, j) e λ a preferência por clientes distantes do depósito. O Algoritmo 1 detalha o procedimento.

Algoritmo 1 Construção por inserção I1 de Solomon (variante de distância)

Entrada: instância; parâmetros $\mu, \lambda, \alpha_1, \alpha_2$

Saída: solução viável S cobrindo todos os clientes

```

1:  $S \leftarrow \emptyset$ ;  $U \leftarrow C$ 
2: enquanto  $U \neq \emptyset$  faça
3:    $u^* \leftarrow \arg \max_{u \in U} d_{0u}$ ;  $r \leftarrow \langle u^* \rangle$ ;  $U \leftarrow U \setminus \{u^*\}$ 
4:   repita
5:      $best \leftarrow \text{nulo}$ ;  $c_2^* \leftarrow -\infty$ 
6:     para todo  $u \in U$  com inserção viável em  $r$  faça
7:       se  $c_2(u) > c_2^*$  então
8:          $c_2^* \leftarrow c_2(u)$ ;  $best \leftarrow (u, \arg \min_{(i,j)} c_1(i, u, j))$ 
9:       fim se
10:    fim para
11:    se  $best \neq \text{nulo}$  então
12:       $\text{insere}(best, r)$ ;  $U \leftarrow U \setminus \{u_{best}\}$ 
13:    fim se
14:  até  $best = \text{nulo}$ 
15:   $S \leftarrow S \cup \{r\}$ 
16: fim enquanto
17: retorne  $S$ 

```

Leitura passo a passo. Na linha 1, a solução começa vazia e o conjunto U de clientes não roteados é inicializado com todos os clientes. Na linha 3, cada nova rota nasce de uma *semente*: o cliente de U mais distante do depósito, escolha que ancora a rota em uma região periférica e tende a reduzir o número total de veículos; a semente é então removida de U . O laço interno (linhas 5–12) faz crescer a rota corrente. Na linha 5, o melhor candidato ($best$) e o melhor valor do critério (c_2^*) são reiniciados. A varredura da linha 6 percorre apenas os clientes de U que admitem inserção viável em r — cada posição candidata é testada em $O(1)$ pela folga Push Forward —, e $c_2(u)$ é avaliado na melhor posição viável, a de custo $c_1^{\min}(u)$; a linha 8 retém o cliente de maior c_2 junto com essa posição. A intuição do critério: λd_{0u} aproxima o custo de servir u em uma viagem dedicada, saindo do depósito só para ele; subtraindo o custo $c_1^{\min}(u)$ de inseri-lo agora, c_2 mede a *economia* de aproveitá-lo na rota corrente. Vence o cliente para o qual essa economia é maior — tipicamente um cliente *distante* do depósito cuja inserção é barata, que seria caro atender depois em uma rota própria.

Na linha 12, o vencedor é inserido na posição retida e removido de U . A rota só se fecha (linha 15) quando nenhuma inserção viável resta, e uma nova é aberta. É essa dinâmica gulosa e sequencial que molda a *solução inicial* de todos os métodos: rotas densas e geograficamente coerentes, porém sem nenhuma reotimização entre rotas — exatamente a lacuna que a busca local explora a seguir.

A cobertura total é garantida estruturalmente: o laço externo só termina quando $U = \emptyset$, e cada iteração roteia ao menos a semente. A viabilidade é garantida porque a varredura da linha 6 só considera inserções que respeitam a folga Push Forward e a capacidade. A Figura 1 ilustra a montagem de uma rota.

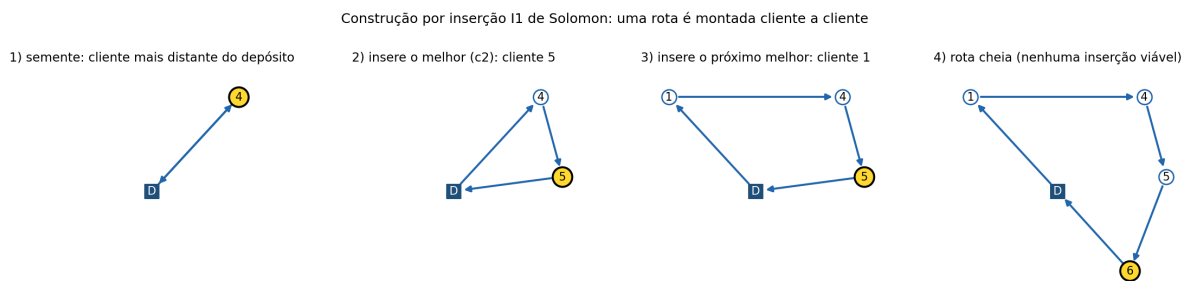


Figura 1 – Construção por inserção I1: a rota começa pela semente (cliente mais distante do depósito) e recebe, a cada passo, o cliente que maximiza c_2 , até nenhuma inserção viável restar. O cliente recém-inserido está em amarelo.

3.4 Vizinhanças e sua escolha

Adotamos um *kit* padronizado de seis operadores, compartilhado por VND, GRASP e Busca Tabu, e listado aqui na ordem em que é explorado — complexidade computacional crescente:

- **2-opt** (intrarota): inverte um subtrecho de uma rota;
- **Swap** (intrarota e entre rotas): troca dois clientes de posição;
- **2-opt*** (entre rotas): troca as caudas de duas rotas (Potvin e Rousseau, 1995);
- **Relocate** (intrarota e entre rotas): remove um cliente e o reinsere em outra posição;
- **Or-opt** (intrarota e entre rotas): move uma cadeia de 2–3 clientes, eventualmente invertida (Or, 1976);
- **Cross-exchange** (entre rotas): troca subtrechos de até 3 clientes entre duas rotas (Taillard et al., 1997).

Todos são operadores clássicos para o VRPTW (Toth e Vigo, 2002). Os seis cobrem os oito movimentos avaliados no relatório parcial: Relocate, Swap e Or-opt implementam

as variantes *intra* e *entre* rotas em uma única varredura, e o “2-opt inter” do parcial corresponde ao 2-opt* de Potvin e Rousseau (1995).

Seleção do subconjunto. Conforme previsto no relatório parcial, os oito movimentos avaliados foram submetidos a seleção por ablação nas instâncias de treino, com Wilcoxon pareado e correção de Holm (Seção 4.3): a remoção de qualquer operador piora o *gap* médio, e nenhum subconjunto próprio supera o conjunto completo. O subconjunto selecionado é, portanto, o próprio conjunto, explorado em ordem de complexidade crescente (Hansen e Mladenović, 2001), com a ordem derivada do custo medido de uma varredura completa.

Por que medir a ordem em vez de derivá-la de uma análise assintótica? Porque, no pior caso, as seis varreduras têm a mesma ordem de complexidade. Cada operador enumera $O(n^2)$ movimentos candidatos (pares de posições, ou de pontos de corte, dentro e entre as rotas) e avalia cada candidato simulando a(s) rota(s) afetada(s) em tempo proporcional ao seu comprimento, $O(n)$ no pior caso — logo, toda varredura completa custa $O(n^3)$. O que distingue os operadores são as *constantes multiplicativas*: o 2-opt só enumera pares dentro de cada rota, o Or-opt multiplica os candidatos pelas quatro variantes de cadeia (2–3 clientes, com e sem inversão) e o Cross-exchange, pelas nove combinações de comprimentos de segmento. A notação assintótica esconde justamente essas constantes; a ordenação foi tomada, então, do custo cronometrado de uma varredura completa (há um teste automatizado que a confere).

Cada operador avalia a viabilidade do movimento por uma simulação completa da(s) rota(s) afetada(s), de modo que nenhum movimento inviável é jamais aplicado (Seção 3.9). A Figura 2 ilustra o efeito de cada um.

3.5 Descida em Vizinhança Variável (VND)

O VND (Hansen e Mladenović, 2001) é o primeiro método que otimiza a Equação (3) diretamente: todo movimento aceito *reduz estritamente* a distância total $D(S)$, e a viabilidade é pré-condição de candidatura (Seção 3.9) — de modo que a trajetória de custos é monotônica decrescente e o método para, por definição, em um ótimo local. Ele percorre as vizinhanças em uma ordem fixa, **da varredura mais barata à mais cara** — ⟨2-opt, Swap, 2-opt*, Relocate, Or-opt, Cross-exchange⟩, a ordem de complexidade medida apresentada na Seção 3.4. Ao encontrar uma melhora (estratégia de *melhor melhora*), aplica-a e reinicia da primeira vizinhança; caso contrário, avança para a próxima. Termina em um ótimo local com relação a *todas* as vizinhanças (Algoritmo 2). É determinístico dada a solução inicial.

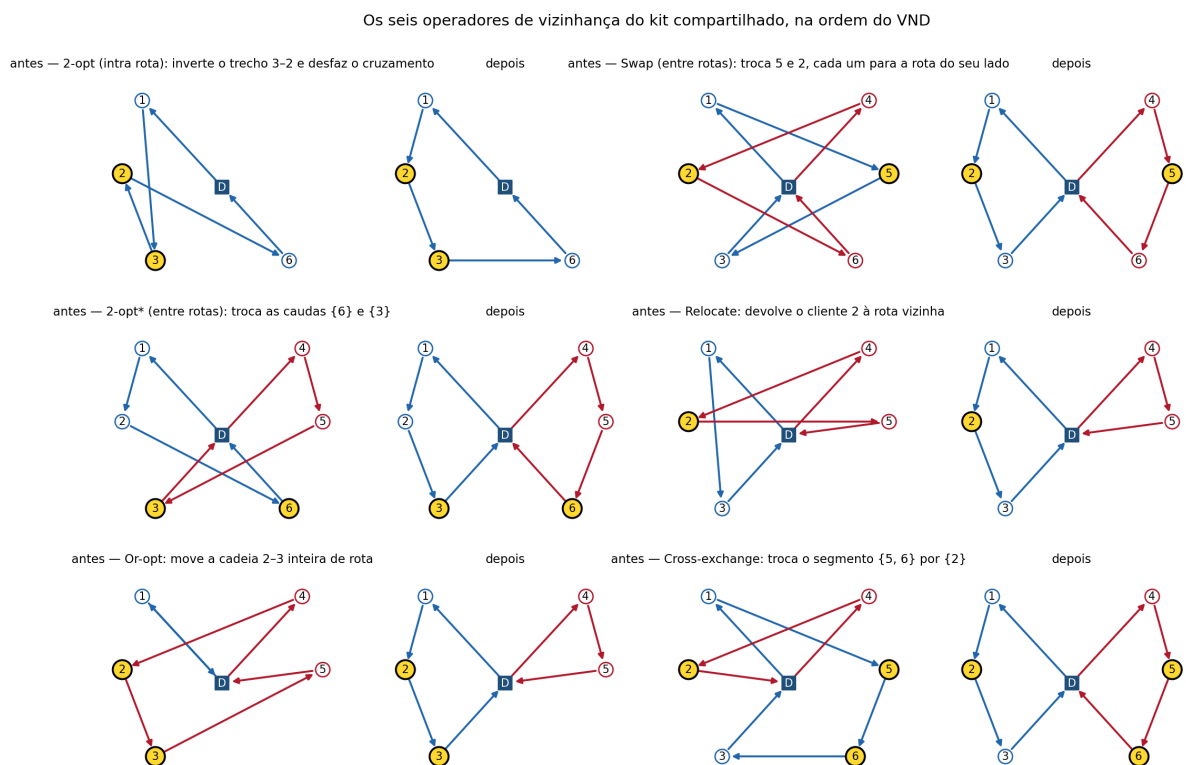


Figura 2 – Os seis operadores de vizinhança do kit (antes/depois), na ordem de exploração do VND, em um exemplo esquemático com duas rotas (azul e vermelha) e o depósito (D); os clientes destacados em amarelo são os que cada movimento desloca. Cada exemplo é um movimento de *melhoria* — o painel “antes” contém a distorção (cruzamento, cliente na rota errada) que o operador conserta —, pois a busca local só aceita movimentos que reduzem a distância.

Algoritmo 2 VND — Descida em Vizinhança Variável

Entrada: solução viável S ; lista ordenada de vizinhanças $N_1, \dots, N_p$

Saída: ótimo local S em relação a todas as vizinhanças

```
1:  $\ell \leftarrow 1$ 
2: enquanto  $\ell \leq p$  faça
3:    $m \leftarrow \text{melhor\_movimento}(N_\ell, S)$             $\triangleright$  viável; nulo se nenhum reduz  $D(S)$ 
4:   se  $m \neq \text{nulo}$  então
5:      $S \leftarrow \text{aplica}(m, S)$ ;  $\ell \leftarrow 1$ 
6:   senão
7:      $\ell \leftarrow \ell + 1$ 
8:   fim se
9: fim enquanto
10: retorne  $S$ 
```

Leitura passo a passo. Na linha 1, o índice ℓ aponta a vizinhança ativa. Na linha 3, a função `melhor_movimento` varre *toda* a vizinhança N_ℓ e retorna o movimento viável de maior redução de distância, ou nulo se nenhum movimento reduz $D(S)$. A estratégia é a de *melhor melhora*: escolhe-se o movimento de maior redução, não o primeiro encontrado. Isso torna a varredura independente da ordem interna de enumeração e, com isso, o resultado determinístico dado o ponto de partida. Se há ganho, a linha 5 aplica o movimento e **reinicia de** $\ell = 1$: voltar à vizinhança mais barata após cada melhoria é o que caracteriza o VND e o que torna a descida sistemática. Sem ganho, a linha 7 passa à vizinhança seguinte; o laço encerra quando nenhuma das p vizinhanças melhora S . A ordem por complexidade tem efeito prático direto: como cada melhoria reinicia da primeira vizinhança, os operadores de varredura barata (2-opt, Swap) absorvem a maior parte dos movimentos e reduzem o trabalho que sobra para os caros (Or-opt, Cross-exchange). É o mecanismo por trás da recomendação de Hansen e Mladenović (2001) de explorar primeiro a vizinhança mais simples. Em termos da *formação da solução*, é aqui que a rota rígida da I1 começa a ser reorganizada: o 2-opt desfaz cruzamentos *dentro* de uma rota, o 2-opt* recombina as *caudas* de duas rotas (podendo até fundi-las), o Relocate corrige clientes mal alocados e o Or-opt e o Cross-exchange transferem trechos *entre* veículos. O ganho quantitativo sobre a I1 é apresentado na Seção 4.5. O VND permanece, porém, preso ao primeiro ótimo local encontrado, e é dessa limitação que partem as meta-heurísticas a seguir.

A Figura 3 ilustra a busca local em ação: cada painel é a solução após um movimento, com a distância caindo monotonicamente até o ótimo local.

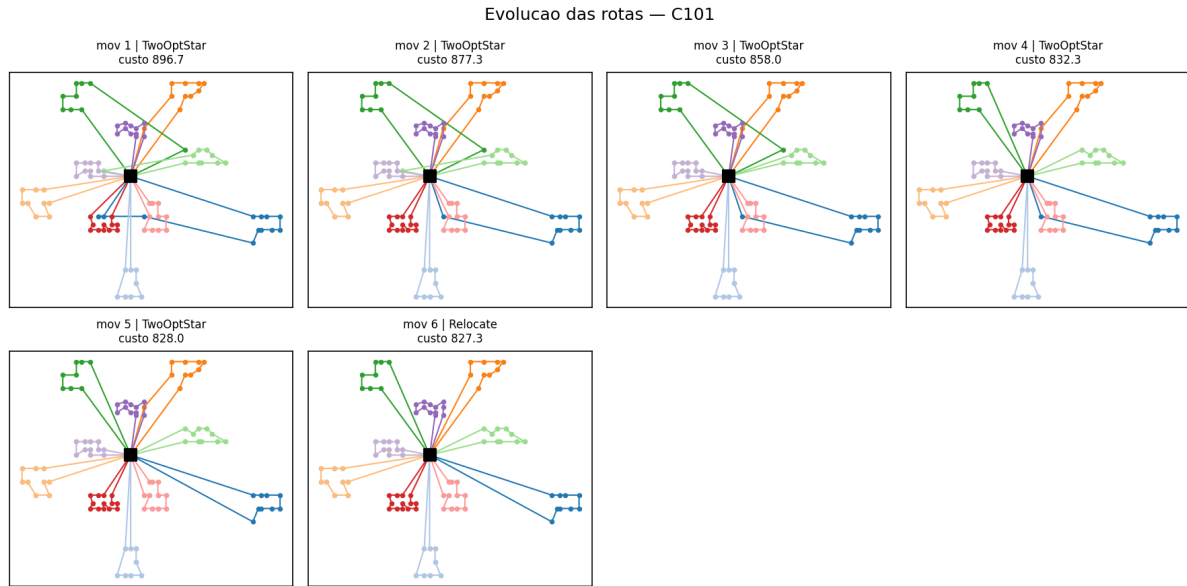


Figura 3 – Evolução das rotas na instância C101 ao longo dos movimentos da busca local (VND): cada painel mostra a solução após um movimento, com o custo decrescendo de 888,0 a 827,3 (o melhor conhecido).

3.6 GRASP

O GRASP (Feo e Resende, 1995; Resende e Ribeiro, 2003) repete um ciclo *construção gulosa-aleatorizada* → *busca local* e guarda a melhor solução. Sua função-objetivo é a mesma Equação (3), tratada por multipartida (*multi-start*): cada iteração produz um ótimo local independente e o resultado é $S^* = \arg \min_i D(S_i)$ sobre todas as iterações — a qualidade final é, portanto, monotonicamente não decrescente no número de iterações (Resende e Ribeiro, 2003). A construção reusa a l1, mas substitui a escolha gulosa do melhor cliente por uma seleção aleatória dentro de uma **Lista Restrita de Candidatos** (RCL) baseada em valor: dado o conjunto de candidatos com valores c_2 , sejam $c^{\max}$ e $c^{\min}$ o maior e o menor; a RCL contém todo candidato com

$$c_2(u) \geq c^{\max} - \alpha (c^{\max} - c^{\min}), \quad \alpha \in [0, 1]. \quad (7)$$

Com $\alpha = 0$ recupera-se a construção gulosa; com $\alpha = 1$, totalmente aleatória. A Figura 4 ilustra a seleção. A busca local é o VND (Algoritmo 2). O Algoritmo 3 resume o método; α é o único parâmetro, calibrado por *irace*.

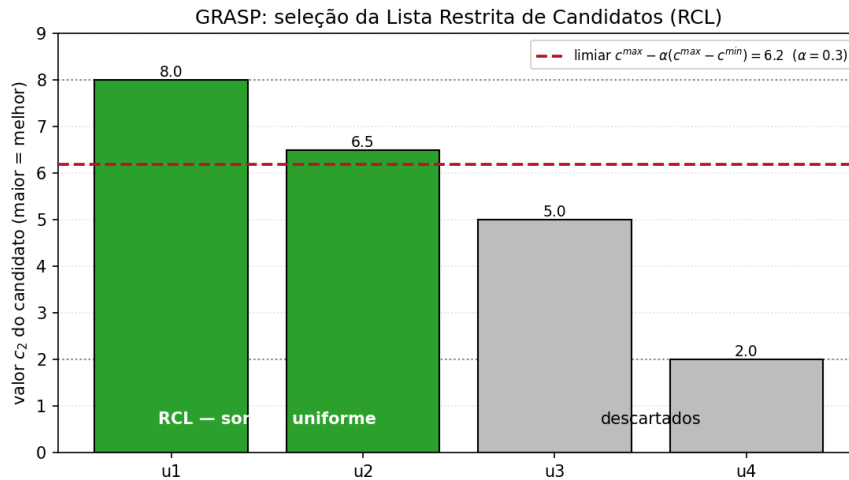


Figura 4 – Seleção da RCL no GRASP: os candidatos com c_2 acima do limiar $c^{\max} - \alpha(c^{\max} - c^{\min})$ (aqui 6,2, com $\alpha = 0,3$) entram na lista (verde) e um deles é sorteado uniformemente; os demais são descartados.

Algoritmo 3 GRASP (Feo & Resende)

Entrada: instância; parâmetro α ; orçamento K (iterações sem melhora)

Saída: melhor solução S^*

```

1:  $S^* \leftarrow I1$   $\triangleright$  partida comum a todos os métodos (Seção 3.3);  $k \leftarrow 0$ 
2: enquanto critério de parada não satisfeito faça
3:    $S \leftarrow \text{ConstruçãoGulosa-Aleatorizada}(\alpha)$ 
4:    $S \leftarrow \text{VND}(S)$ 
5:   se distância( $S$ ) < distância( $S^*$ ) então
6:      $S^* \leftarrow S$ ;  $k \leftarrow 0$ 
7:   senão
8:      $k \leftarrow k + 1$ 
9:   fim se
10:  se  $k \geq K$  então
11:    interrompa
12:  fim se
13: fim enquanto
14: retorne  $S^*$ 

```

Leitura passo a passo. Na linha 1, o algoritmo difere da forma de livro-texto, que parte de incumbente vazio: aqui, o *incumbente* (a melhor solução encontrada até o momento, que será devolvida ao final) é inicializado com a solução da I1 pura, a mesma partida entregue ao VND e à Busca Tabu. Isso garante, por construção, que o resultado do GRASP nunca seja pior que a I1, e põe todos os métodos partindo do mesmo ponto. Cada iteração é então um ciclo *construção* $\rightarrow$ *busca local*: a linha 3 gera uma solução nova com aleatorização controlada por α (a RCL da Figura 4), e a linha 4 a conduz a um ótimo local pelo VND (Algoritmo 2). A aleatorização age apenas na escolha de qual

cliente inserir; a regra de semente e o critério c_2 permanecem os da I1, e é por isso que, com $\alpha = 0$, o GRASP degenera na construção da I1 — propriedade verificada por teste automatizado nas 56 instâncias: em 47 a solução é idêntica à da I1 e, nas 9 restantes, o teste confirma que a diferença decorre exclusivamente de *empates* no critério c_2 , caso em que a lista restrita contém mais de um candidato e a escolha passa pelo gerador aleatório. A linha 6 retém a melhor solução já vista e zera o contador de estagnação; a linha 8 o incrementa quando não há ganho; e a linha 11 encerra após K iterações sem melhora. O acréscimo do GRASP é a *diversificação estruturada*: reconstruindo as rotas do zero a cada iteração, ele rompe a dependência da semente única da I1 (cada RCL produz uma topologia de rotas diferente), e o resultado final é a melhor entre centenas de descidas independentes.

3.7 GRASP Reativo

O GRASP Reativo (Prais e Ribeiro, 2000) otimiza a mesma função-objetivo do GRASP — Equação (3), por multipartida — diferindo apenas em *como* o parâmetro de aleatorização é escolhido: em vez de um α fixo, mantém-se um conjunto discreto $\{\alpha_1, \dots, \alpha_g\}$ (grade 0,05, 0,10, $\dots$, 0,50) com probabilidades p_i de seleção. A cada bloco de *block* iterações, as probabilidades são atualizadas de modo que valores de α que produziram soluções melhores fiquem mais prováveis — o expoente δ regula a agressividade desse aprendizado (δ baixo mantém as probabilidades próximas do uniforme; δ alto concentra rapidamente no melhor α):

$$q_i = \left(\frac{Z^*}{\bar{Z}_i} \right)^\delta, \quad p_i = \frac{q_i}{\sum_j q_j}, \quad (8)$$

em que Z^* é o melhor custo encontrado e $\bar{Z}_i$ o custo médio (acumulado ao longo de toda a execução) das soluções obtidas com α_i . Os parâmetros calibrados por *irace* são δ (expoente) e *block_frac*: na implementação, o tamanho de bloco é derivado como $block = block_frac \cdot K$, para que o número de reponderações independa do orçamento (Seção 3.11). O Algoritmo 4 detalha o método; nas primeiras g iterações cada α é experimentado uma vez (*aquecimento*) antes de a amostragem por probabilidade começar. Como $block \geq 16 > g = 10$ em toda a faixa calibrada, a primeira reponderação só ocorre após o fim do aquecimento, quando todo α_i já tem $n_i \geq 1$ — as linhas 20–21 nunca encontram $n_i = 0$.

Algoritmo 4 GRASP Reativo (Prais & Ribeiro)

Entrada: grade $\{\alpha_1, \dots, \alpha_g\}$; expoente δ ; tamanho de bloco $block$; orçamento K

Saída: melhor solução S^*

```

1:  $p_i \leftarrow 1/g$ ,  $\Sigma_i \leftarrow 0$ ,  $n_i \leftarrow 0$  para todo  $i$ ;  $S^* \leftarrow \text{I1}$ ;  $k \leftarrow 0$ ;  $it \leftarrow 0$ 
2: enquanto critério de parada não satisfeito faça
3:   se  $it < g$  então
4:      $j \leftarrow it + 1$   $\triangleright$  aquecimento: cada  $\alpha$  uma vez
5:   senão
6:      $j \leftarrow$  amostra de  $\{1, \dots, g\}$  segundo as probabilidades  $p$ 
7:   fim se
8:    $S \leftarrow \text{VND}(\text{ConstruçãoGulosa-Aleatorizada}(\alpha_j))$ 
9:    $c \leftarrow \text{distância}(S)$ ;  $\Sigma_j \leftarrow \Sigma_j + c$ ;  $n_j \leftarrow n_j + 1$ 
10:  se  $c < \text{distância}(S^*)$  então
11:     $S^* \leftarrow S$ ;  $k \leftarrow 0$ 
12:  senão
13:     $k \leftarrow k + 1$ 
14:  fim se
15:   $it \leftarrow it + 1$ 
16:  se  $k \geq K$  então
17:    interrompa
18:  fim se
19:  se  $it \bmod block = 0$  então
20:     $Z^* \leftarrow \text{distância}(S^*)$ ;  $\bar{Z}_i \leftarrow \Sigma_i / n_i$ ,  $q_i \leftarrow (Z^* / \bar{Z}_i)^\delta$  para todo  $i$ 
21:     $p_i \leftarrow q_i / \sum_l q_l$  para todo  $i$ 
22:  fim se
23: fim enquanto
24: retorne  $S^*$ 

```

Leitura passo a passo. O reativo difere do GRASP por *aprender* qual α funciona melhor em cada instância. Nas linhas 4–6, escolhe-se o α : nas primeiras g iterações cada valor é testado uma vez (aquecimento, para que nenhum α comece com vantagem espúria); depois, α é sorteado conforme as probabilidades p . Na linha 9, acumulam-se a soma e a contagem dos custos obtidos por cada α e, a cada $block$ iterações, as linhas 20–21 recalculam as probabilidades pela razão $q_i = (Z^* / \bar{Z}_i)^\delta$, tomando Z^* como a distância do incumbente: valores de α que produziram soluções melhores tornam-se mais prováveis. O efeito prático é ajustar automaticamente o equilíbrio guloso↔aleatório ao perfil da instância. Resta saber se esse aprendizado se traduz em vantagem mensurável sobre o GRASP de α fixo; a análise estatística da Seção 4.5 trata exatamente dessa pergunta.

3.8 Busca Tabu

Adotamos a forma clássica e mais simples da Busca Tabu (Glover, 1989, 1990): a cada iteração, move-se para o *melhor vizinho admissível* sobre o kit de vizinhanças, aceitando, se necessário, movimentos que pioram a solução, para escapar de ótimos locais. A função-objetivo é a mesma Equação (3), com uma distinção operacional: a solução *corrente* pode piorar de um passo para o outro; quem realiza a minimização é o *incumbente* S^* , que retém o menor $D(S)$ visitado ao longo da trajetória única do método. Um movimento é *tabu* se algum dos seus clientes-atributo foi movido nas últimas *tenure* iterações, a menos que satisfaça o critério de *aspiração por objetivo* (produzir um novo melhor global). Adotamos também a *aspiração por omissão* (Glover e Laguna, 1997): se **todos** os movimentos disponíveis estiverem tabu, revoga-se o status do grupo de clientes com prazo de expiração mais próximo e a varredura é refeita, em vez de encerrar a busca — sem essa salvaguarda, a lista tabu pode esgotar a vizinhança e truncar a execução muito antes do critério de parada. O atributo tabu é, precisamente, o *primeiro cliente de cada segmento movido*: o cliente relocado no Relocate, os dois clientes trocados no Swap, a cabeça da cadeia no Or-opt, os extremos do trecho invertido no 2-opt e o primeiro cliente de cada cauda ou segmento trocado no 2-opt* e no Cross-exchange. É uma simplificação deliberada (em vez do arco, ou de todos os clientes tocados), mantida para preservar a forma “de livro-texto” com custo de memória mínimo. Não há religação de caminhos (*path relinking*), intensificação ou diversificação por frequência. O único parâmetro é *tenure*, calibrado por *irace* (Algoritmo 5).

Algoritmo 5 Busca Tabu (Glover, forma clássica)

Entrada: solução inicial S (da l1); *tenure*; orçamento K

Saída: melhor solução S^*

```
1:  $S^* \leftarrow S$ ;  $k \leftarrow 0$ 
2: enquanto critério de parada não satisfeito faça
3:    $m \leftarrow \text{melhor\_movimento\_admissível}(N, S)$    ▷ viável; não tabu, ou tabu com
   aspiração
4:   se  $m = \text{nulo}$  então
5:     se nenhum cliente está tabu então
6:       interrompa                                     ▷ não existe movimento viável algum
7:     fim se
8:     revoga o status tabu do grupo com menor prazo de expiração
9:     volte à linha 3   ▷ aspiração por omissão; a varredura é refeita sem consumir
   iteração
10:  fim se
11:   $S \leftarrow \text{aplica}(m, S)$ ; marque o(s) cliente(s)-atributo de  $m$  como tabu por tenure
   iterações
12:  se  $\text{distância}(S) < \text{distância}(S^*)$  então
13:     $S^* \leftarrow S$ ;  $k \leftarrow 0$ 
14:  senão
15:     $k \leftarrow k + 1$ 
16:  fim se
17:  se  $k \geq K$  então
18:    interrompa
19:  fim se
20: fim enquanto
21: retorne  $S^*$ 
```

Leitura passo a passo. A cada iteração, a linha 3 varre todo o kit de vizinhanças e escolhe o melhor movimento *admissível* — não tabu ou, se tabu, que satisfaça a aspiração por objetivo. Diferentemente do VND, a Tabu **não** exige que o movimento melhore: a linha 11 pode aplicar uma piora controlada, e é assim que a busca escapa de ótimos locais. O movimento torna tabu, por *tenure* iterações, o(s) seu(s) cliente(s)-atributo (memória de curto prazo), o que impede a busca de reverter movimentos recentes e ciclar. O *tenure* regula o compromisso central do método: valores pequenos liberam os clientes cedo e intensificam a busca na região atual; valores grandes prolongam as proibições e forçam a exploração de regiões novas. Quando a lista tabu chega a bloquear *todos* os movimentos — situação comum com *tenure* alto em instâncias pequenas —, as linhas 8–9 aplicam a aspiração por omissão: expiram o grupo de proibições mais antigo e repetem a varredura, sem consumir iteração. O caso-limite da linha 6 encerra a busca apenas se não existir movimento viável algum, tabu ou não — fora dele, a busca só termina pelo critério de parada. A linha 13 guarda o melhor

já visto. No conjunto, a Tabu percorre uma *única* trajetória contínua e refina a estrutura herdada da I1 sem reinícios; o GRASP, ao contrário, explora múltiplas estruturas independentes.

3.9 Critérios de aceitação

Os cinco métodos diferem essencialmente *no que aceitam* a cada passo. Tornamos isso explícito:

- **Filtro de viabilidade (comum a todos).** Um movimento só é *candidato* se a(s) rota(s) resultante(s) satisfaz(em) a Equação (1). Movimentos inviáveis são descartados *antes* da comparação de custo. Esse filtro é o que garante o invariante da Seção 2.3.
- **VND e GRASP:** aceitam um movimento apenas se ele *reduz estritamente* a distância (melhora). O GRASP, além disso, guarda a melhor solução ao longo das reinicializações.
- **Busca Tabu:** aceita, a cada iteração, o *melhor vizinho admissível*, mesmo que piore a solução corrente; a regra tabu proíbe reverter movimentos recentes (memória de curto prazo), e a aspiração libera um movimento tabu se ele gerar um novo melhor global.
- **RCL (GRASP):** na construção, um candidato é aceito na lista restrita se seu valor c_2 está dentro de uma fração α do intervalo $[c^{\min}, c^{\max}]$; a escolha final é uniforme sobre a RCL.

3.10 Exemplos ilustrativos do funcionamento

Para tornar concretos os mecanismos centrais, apresentamos três exemplos mínimos e verificáveis.

(a) Critério Push Forward (viabilidade de inserção em $O(1)$). Considere o depósito $D = (0, 0)$, horizonte $[0, 100]$, e a rota $\langle D, A, B, D \rangle$ com $A = (10, 0)$, janela $[5, 40]$, serviço 5; $B = (30, 0)$, janela $[20, 60]$, serviço 5; capacidade $Q = 50$ e demandas 10. As distâncias são $d_{DA} = 10$, $d_{AB} = 20$, $d_{BD} = 30$. A Tabela 1 mostra o cronograma: partindo de D em $t = 0$, todos os inícios respeitam as janelas e o retorno ocorre em $70 \leq 100$. A *folga (slack)* de cada cliente é a margem de adiamento do seu início sem violar nada à frente: $\text{slack}(B) = \min(60 - 35, 100 - 70) = 25$ e $\text{slack}(A) = \min(40 - 10, 0 + 25) = 25$.

Tabela 1 – Cronograma da rota $\langle D, A, B, D \rangle$ do exemplo (a).

nó	chegada	início	partida	janela
<i>A</i>	10	10	15	[5, 40]
<i>B</i>	35	35	40	[20, 60]
<i>D</i> (retorno)	70	—	—	[0, 100]

Suponha inserir $C = (15, 0)$, janela $[10, 50]$, serviço 5, demanda 10, entre *A* e *B*. Verifica-se em $O(1)$: a capacidade $(20 + 10 \leq 50)$; o início de *C*, $\max(15 + 5, 10) = 20 \leq 50$; e o novo início de *B*, $\max(25 + 15, 20) = 40$, um *deslocamento* de $40 - 35 = 5$. Como $5 \leq \text{slack}(B) = 25$, a inserção é **viável** — sem reavaliar a rota inteira. É exatamente esse teste que cada operador de melhoria aplica antes de aceitar um movimento.

(b) Lista Restrita de Candidatos (GRASP). Suponha quatro candidatos com valores c_2 : $u_1=8,0$, $u_2=6,5$, $u_3=5,0$, $u_4=2,0$, logo $c^{\max}=8,0$ e $c^{\min}=2,0$. Com $\alpha = 0,3$, o limiar é $8,0 - 0,3 \cdot (8,0 - 2,0) = 6,2$; assim $\text{RCL} = \{u_1, u_2\}$ e o próximo cliente é sorteado uniformemente entre os dois. Com $\alpha = 0$ a RCL seria $\{u_1\}$ (guloso puro); com $\alpha = 1$ conteria todos (aleatório puro). É esse parâmetro que regula o equilíbrio guloso–aleatório, calibrado por *irace*.

(c) Uma iteração da Busca Tabu. Da solução corrente, a varredura encontra como melhor movimento *viável e admissível* um Relocate do cliente 7, com variação de distância $\Delta = -3,1$. Aplica-se o movimento e o cliente 7 torna-se *tabu* por *tenure* iterações: nesse período, qualquer movimento que toque o cliente 7 é proibido — exceto se produzir um custo melhor que o melhor global já visto (aspiração). Assim, a busca aceita pioras controladas para escapar de ótimos locais, mas evita ciclar revertendo movimentos recentes.

3.11 Parametrização

Cada parâmetro deste estudo pertence a uma de três categorias, e a categoria determina como seu valor foi obtido. Parâmetros **fixos** definem o terreno comum da comparação — a solução inicial, o conjunto de vizinhanças, a convenção de distância — e por isso não podem ser ajustados por método: calibrá-los daria a um algoritmo uma condição de partida diferente da dos demais. Parâmetros **calibrados** são os de qualidade intrínsecos a cada meta-heurística, ajustados pelo *irace* sobre as instâncias de treino (Seção 3.13); seus valores finais ficam registrados em `experiments/config/tuned.json` e são citados neste texto apenas onde a discussão os exige (Seção 4.4). Parâmetros **declarados** são orçamentos computacionais, para os quais não existe valor ótimo a

descobrir (Seção 3.12). A Tabela 2 consolida as três categorias.

Tabela 2 – Parâmetros dos algoritmos: papel, valor ou intervalo de busca, e origem de cada um.

Algoritmo	Parâmetro	Papel	Valor / intervalo	Origem
I1	μ	economia do arco removido em c_{11}	1,0	fixo
I1	λ	peso da distância ao depósito em c_2	2,0	fixo
I1	α_1, α_2	mistura distância $\times$ tempo em c_1	1,0 / 0,0	fixo
I1	semente	cliente que abre cada rota	mais distante	fixo
VND	ordem	sequência de exploração das vizinhanças	compl. medida	fixo
GRASP	α	largura da RCL (guloso $\leftrightarrow$ aleatório)	(0; 0,5)	calibrado
Reativo	δ	expoente da reponderação das probabilidades	(0,5; 3,0)	calibrado
Reativo	<i>block_frac</i>	cadência de reponderação, fração de K	(0,02; 0,40)	calibrado
Reativo	grade de α	valores candidatos ao sorteio	0,05–0,50 (10)	fixo
Tabu	<i>tenure</i>	duração da proibição (memória curta)	(5; 80)	calibrado
todos	K	iterações sem melhora (parada)	800	declarado
todos	teto de tempo	salvaguarda de execução	600 s	declarado

Quatro escolhas da tabela merecem justificativa explícita. Os parâmetros da I1 usam a configuração de distância de Solomon (1987) ($\alpha_2 = 0$), coerente com o objetivo DIMACS, e são fixos porque a I1 é a partida comum de todos os métodos. O intervalo de α do GRASP para em 0,5 porque, acima disso, a construção degenera em sorteio quase uniforme e perde a estrutura da I1 que a busca local aproveita. O *block_frac* do Reativo é expresso como fração de K , e não em iterações absolutas, para que o número de reponderações independa do orçamento — com bloco absoluto, um orçamento pequeno pode encerrar a execução antes da primeira reponderação, e o mecanismo reativo simplesmente não opera. O teto do *tenure* foi definido após uma primeira rodada de calibração em que o vencedor caiu na fronteira superior do intervalo então vigente; um vencedor de fronteira indica intervalo curto, e a resposta metodológica é alargá-lo e repetir a corrida em vez de reportar o valor de borda como ótimo.

3.12 Critério de parada

A I1 e o VND são determinísticos e terminam naturalmente (a I1 quando todos os clientes estão roteados; o VND em um ótimo local). Para os métodos iterativos (GRASP e GRASP Reativo, estocásticos; Busca Tabu, determinística) adotamos como critério primário o **número de iterações consecutivas sem melhora do incumbente (K)**, na unidade natural de iteração de cada método (uma iteração do GRASP é uma construção seguida de VND; uma iteração da Tabu é um movimento). O tempo de relógio é registrado e reportado, mas serve apenas como teto de segurança. Essa escolha favorece a **reprodutibilidade** (uma parada por tempo depende da máquina), ao custo de que uma “iteração” não representa o mesmo esforço entre métodos; por isso re-

portamos também o tempo e o número de iterações de cada execução, e as curvas tempo-alvo (TTT) (Aiex et al., 2007), o que torna o esforço comparável. A *competição* DIMACS em si pontua por tempo de relógio padronizado e integral primal (DIMACS, 2022). Como este trabalho é um *estudo comparativo*, não uma submissão à competição, adotamos a parada por iterações sem melhora em favor da reprodutibilidade, mantendo a mesma convenção de distância e objetivo do desafio.

3.13 Calibração de parâmetros (irace)

A calibração é feita com o pacote *irace* (López-Ibáñez et al., 2016), padrão de fato para configuração automática de algoritmos. Seu mecanismo é a *corrida (racing)*: as configurações candidatas são avaliadas instância a instância e, a cada rodada, um teste estatístico elimina as que já se mostram inferiores — concentrando o orçamento de avaliações nas candidatas sobreviventes, em vez de gastá-lo uniformemente. Para evitar superajuste, as 56 instâncias são divididas em **treino** (28) e **teste** (28), estratificadas por família (C/R/RC) e tipo (1/2); o *irace* usa apenas o treino, e os resultados são reportados no teste e no conjunto completo. Adotamos um desenho **desacoplado**: o critério de parada K é *declarado* (a seguir), e o *irace* calibra *apenas* os hiperparâmetros de qualidade de cada método — GRASP: α ; Reativo: δ e *block_frac*; Tabu: *tenure*. Os demais elementos (parâmetros da I1, conjunto de vizinhanças, número de execuções) são *fixados* por justiça. A função-objetivo do *irace* é o *gap* de distância ao melhor valor conhecido, que normaliza escalas entre instâncias. A justificativa de cada faixa de busca e da separação “calibrar vs. fixar” está documentada no arquivo TUNING_RATIONALE.

Fixação de K . O critério de parada K é um *orçamento*, não um parâmetro de qualidade: mais iterações sem melhora nunca pioram o gap. Essa monotonicidade tem uma consequência que precisa ser enunciada, porque ela decide o procedimento: **não existe valor ótimo de K a ser descoberto**. Um orçamento monótono não pode ser escolhido minimizando o objetivo — calibrá-lo pelo gap, junto aos parâmetros de qualidade, o levaria sempre ao teto do intervalo de busca, confundindo *orçamento* com *qualidade*. É também o motivo pelo qual a literatura de GRASP e de Busca Tabu não deriva K : ela o *declara* (Feo e Resende, 1995; Resende e Ribeiro, 2003; Prais e Ribeiro, 2000).

Adotamos, portanto, K como **orçamento computacional declarado, uniforme entre os métodos** — $K_{\text{GRASP}} = K_{\text{Reativo}} = K_{\text{Tabu}} = 800$ — de modo que a regra de parada seja a mesma para todos. A Tabela 3 mostra o que cada orçamento entrega (gap médio nas instâncias de treino) e o que custa (tempo mediano, medido sem concorrência em

seis instâncias-amostra, uma por família e tipo). Em $K = 800$ a execução mediana leva 15,6 s no GRASP, 22,7 s no Reativo e 6,8 s na Busca Tabu, com pior caso de 76 s — com ampla margem em relação ao teto de segurança de 600 s. Dobrar o orçamento para $K = 1600$ reduz o gap do GRASP em 0,19 ponto percentual (pp) e custa $1,8\times$ o tempo ($2,3\times$ no Reativo).

A escolha não influencia nenhuma conclusão do estudo, e verificamos isso diretamente: **o ordenamento dos métodos e o resultado dos testes par a par são idênticos em toda a faixa medida**, de $K = 50$ a $K = 3200$ — uma variação de $64\times$ (Apêndice B). A contrapartida de um K uniforme é que uma “iteração” não custa o mesmo em cada método; por isso reportamos o tempo, o número de iterações e as curvas tempo-alvo (TTT), e complementamos a leitura com a comparação sob tempo igual e a integral primal.

Tabela 3 – Efeito do orçamento de parada. *Gap*: distância percentual média ao melhor valor conhecido, nas 28 instâncias de treino. *Tempo*: mediana por execução, medida sem concorrência em seis instâncias-amostra (uma por família e tipo). O valor adotado ($K = 800$) está em negrito. O gap decresce ao longo de toda a faixa medida, sem platô — razão pela qual K é declarado como orçamento computacional, e não derivado de um ponto de convergência.

K	gap médio (%)			tempo mediano (s)		
	GRASP	Reativo	Tabu	GRASP	Reativo	Tabu
50	3,49	3,11	5,99	—	—	—
100	3,15	2,71	5,47	—	—	—
200	2,39	2,27	4,68	4,6	6,9	1,6
400	2,17	1,93	4,28	9,5	11,8	3,9
800	1,86	1,65	4,04	15,6	22,7	6,8
1600	1,67	1,52	3,64	27,9	52,5	13,9
3200	1,47	1,20	3,28	—	—	—

3.14 Comparação estatística

Cada algoritmo estocástico é executado 30 vezes por instância, com sementes independentes registradas — tamanho de amostra usual em comparações de meta-heurísticas, suficiente para estimativas estáveis de média e dispersão por instância; os determinísticos (I1, VND, Busca Tabu) executam uma vez, por não possuírem fonte de aleatoriedade. Para comparar os métodos sobre as 56 instâncias usamos o teste não paramétrico de **Friedman** sobre os *ranks* por instância: em cada instância, os k métodos são ordenados do menor ao maior gap médio, e o *rank* de um método é sua posição nessa ordenação (1 = melhor; empates recebem a média das posições). O teste pergunta se os *ranks* médios diferem mais do que o acaso explicaria, e é seguido

do pós-teste de **Nemenyi** (todas as comparações par a par), conforme Demšar (2006). A estatística de Friedman, *com correção para empates* (frequentes, pois na família C vários métodos atingem o ótimo), é

$$\chi_F^2 = \frac{1}{C} \cdot \frac{12N}{k(k+1)} \left[\sum_{j=1}^k R_j^2 - \frac{k(k+1)^2}{4} \right], \quad C = 1 - \frac{\sum_g (t_g^3 - t_g)}{N(k^3 - k)}, \quad (9)$$

com N instâncias, k algoritmos, R_j o *rank* médio do algoritmo j (rank 1 = melhor) e t_g o tamanho de cada grupo de empate. A correção para empates é relevante neste conjunto: na família C vários métodos atingem o mesmo custo, e ignorá-la subestimaria a estatística. É o cálculo realizado pela implementação (`scipy.stats.friedmanchisquare`). No pós-teste de Nemenyi, dois algoritmos diferem significativamente se seus ranks médios distam mais que a *diferença crítica*

$$CD = q_\alpha \sqrt{\frac{k(k+1)}{6N}}, \quad (10)$$

em que q_α é o valor crítico do *range* estudentizado (dividido por $\sqrt{2}$); para $k = 5$ e $\alpha = 0,05$ ($q_\alpha = 2,728$), tem-se $CD = 0,815$ com $N = 56$. Os resultados são sintetizados em um *diagrama de diferença crítica* (CD). O teste de Friedman é apropriado por ser pareado (mesmas instâncias para todos os métodos) e robusto a distribuições não normais dos gaps.

O pós-teste de Nemenyi opera sobre *ranks médios*, e uma limitação conhecida dessa família de procedimentos é que a conclusão sobre um par de métodos depende também do desempenho dos demais métodos incluídos na comparação (Benavoli et al., 2016). Por isso, cada comparação par a par de interesse é adicionalmente examinada pelo teste de **Wilcoxon pareado** sobre os gaps por instância, com correção de **Holm** para as múltiplas comparações (García e Herrera, 2008) — um procedimento que usa apenas os dados do próprio par. A implementação do Wilcoxon usa a aproximação normal da estatística, com correção de continuidade e de empates na variância, adequada ao tamanho de amostra ($N = 28$ ou 56); zeros são descartados pelo método clássico de Wilcoxon. As duas leituras são reportadas; conclusões afirmadas no texto exigem concordância entre ambas.

3.15 Ambiente computacional e reprodutibilidade

Os experimentos foram conduzidos em uma única máquina — Intel® Core™ Ultra 9 285K (24 *threads*), Ubuntu 24.04 LTS (kernel 6.17), com o solver compilado por g++ 13.3

e `-std=c++20 -O2`. Cada execução é reproduzível a partir da semente registrada, pelo protocolo portátil descrito na Seção 3.2. A convenção de distância e a corretude do avaliador são verificadas contra soluções de referência (Seção 4.1).

4 Resultados e Discussão

O desempenho é medido pelo *gap* percentual de distância ao melhor conhecido,

$$\text{gap}(\%) = 100 \cdot \frac{d - d^{\text{bk}}}{d^{\text{bk}}}, \quad (11)$$

onde d é a distância obtida e d^{bk} a da solução de referência (Seção 4.7); o *gap* normaliza as escalas entre instâncias de tamanhos e densidades diferentes. Reportamos o *gap médio* (sobre as 30 execuções dos métodos estocásticos) e o *melhor*.

4.1 Verificação da convenção de distância

Antes de qualquer comparação, validamos a implementação reproduzindo o custo das 56 soluções de referência. Recomputando a distância de cada solução de referência sob a convenção truncada, obtivemos os custos declarados em **todas as 56 instâncias** (diferença absoluta $\leq 0,05$). Esse teste é um *portão*: nenhum resultado é considerado antes de ele passar.

4.2 Viabilidade das soluções

Em todas as 3 528 execuções do estudo — 56 instâncias $\times$ (1 da I1 + 1 do VND + 1 da Tabu + 30 do GRASP + 30 do Reativo); a contagem por método é registrada em `run_counts.csv` —, o verificador independente não acusou **nenhuma** solução inviável, e nenhum *gap* negativo (distância inferior à referência) ocorreu. Isso confirma, empiricamente, o invariante da Seção 2.3: os algoritmos de melhoria formam e modificam rotas *sem jamais violar* cobertura, capacidade ou janelas de tempo.

4.3 Ablação do conjunto de vizinhanças

A Tabela 4 quantifica a contribuição de cada operador por *ablação leave-one-out*: o VND é executado com o kit completo e, em seguida, seis vezes com um operador removido de cada vez, sempre sobre as mesmas 28 instâncias de treino e a partir da mesma solução inicial I1 — de modo que a única diferença entre as linhas é a presença do operador. Três métricas acompanham cada remoção: o **gap médio** resultante, o

placar por instância (em quantas a remoção piora, melhora ou empata) e o *p*-valor do teste de Wilcoxon pareado com correção de Holm para múltiplas comparações.

Tabela 4 – Ablação *leave-one-out* do conjunto de vizinhanças (VND, 28 instâncias de treino): efeito de remover cada operador do kit completo, com o placar por instância e o *p* de Wilcoxon pareado sob correção de Holm. A remoção de qualquer operador piora o gap médio; a última coluna mostra que o teste tem poder limitado para os operadores que afetam poucas instâncias.

Configuração	Gap médio (%)	Δ (pp)	piora/melhora/empata	<i>p</i> (Holm)	signif.
Kit completo (6 operadores)	8,25	—	—	—	—
sem Relocate	11,40	+3,15	23/4/1	0,0015	sim
sem 2-opt*	9,67	+1,42	16/6/6	0,1745	não
sem 2-opt intra	9,25	+1,00	8/8/12	0,7476	não
sem Swap	9,23	+0,98	13/8/7	0,5349	não
sem Or-opt	9,01	+0,76	12/1/15	0,0216	sim
sem Cross-exchange	8,75	+0,50	4/0/24	0,5349	não

A remoção de qualquer operador piora o gap médio, o que sustenta a adoção do kit completo como resultado do procedimento de seleção (Seção 3.4). O Relocate é o operador dominante: sua remoção custa +3,15 pp e piora o gap em 23 das 28 instâncias ($p = 0,0015$). Em seguida vem o 2-opt* (+1,42 pp), cuja ausência pesa sobretudo nas famílias R e RC, onde a recombinação de caudas entre rotas é mais valiosa. Note-se que apenas Relocate e Or-opt atingem significância estatística, e que isso não autoriza remover os demais. O Cross-exchange ilustra o porquê: ele altera o resultado em somente 4 das 28 instâncias (as outras 24 empatam exatamente), e com quatro observações não nulas o teste de Wilcoxon não alcança $p < 0,05$ nem no melhor cenário possível — a falta de significância aqui é falta de *poder* do teste, não indício de que o operador seja dispensável. A busca exaustiva sobre os 63 subconjuntos, que não depende de teste de hipótese, aponta na mesma direção: os melhores subconjuntos por família diferem entre si, e sua união é o conjunto completo.

4.4 A calibração valeu a pena?

Uma seção de calibração precisa responder à pergunta que lhe dá sentido: os parâmetros calibrados são de fato melhores que os valores clássicos da literatura? O protocolo foi declarado *antes* da corrida do *irace*: o calibrado é aceito, por cenário, se supera o clássico no conjunto de teste com Wilcoxon pareado a 5%; caso contrário, adota-se o valor clássico e reporta-se que a calibração não rendeu ganho demonstrável. A Tabela 5 traz o confronto.

Tabela 5 – A calibração valeu a pena? Parâmetros clássicos da literatura ($\alpha=0,30$; $\delta=1,0$, $block_frac=0,1$; $tenure=15$) contra os calibrados pelo *irace*, nas 28 instâncias de teste — nunca vistas pela calibração —, com 30 sementes por método estocástico (a Busca Tabu, determinística, executa uma vez por configuração) e o mesmo $K=800$. Ganho positivo = calibrado melhor; p do teste de Wilcoxon pareado.

Cenário	Gap clássico (%)	Gap calibrado (%)	Ganho (pp)	melhor/pior/empate	p
GRASP	2,01	2,03	-0,02	9/11/8	0,6316
GRASP reativo	2,03	2,00	+0,03	14/6/8	0,0496
Busca Tabu	5,30	4,04	+1,26	13/6/9	0,0431

O resultado difere por método. Na **Busca Tabu**, calibrar rendeu +1,26 pp ($p=0,043$): o *tenure* calibrado (49) está bem acima do clássico (15), indicando que, sob o kit de seis vizinhanças — cuja vizinhança conjunta é grande —, proibições mais longas são necessárias para forçar a exploração. No **GRASP**, o calibrado ($\alpha=0,22$) não superou o clássico ($\alpha=0,30$): a diferença é indistinguível de ruído ($p=0,63$) e, nesse caso, o protocolo pré-declarado determina adotar o clássico — por isso o estudo principal usa $\alpha=0,30$. Esse resultado negativo é informativo: **o GRASP é insensível a α na faixa 0,2–0,3** nestas instâncias, o que é coerente com o empate observado entre o GRASP fixo e o Reativo (se houvesse um α nitidamente melhor, o mecanismo reativo teria vantagem em encontrá-lo). No **Reativo**, o ganho é positivo e limítrofe (+0,03 pp, $p=0,0496$): o valor calibrado foi mantido por cumprir o critério, mas o que os dados sustentam é “não pior que o clássico, com indício fraco de vantagem”. O superajuste da calibração é pequeno: a diferença de gap entre treino e teste fica entre 0,10 e 0,24 pp nos três métodos.

4.5 Comparação de desempenho

Sob o critério de parada por iterações sem melhora ($K=800$, uniforme; Seção 3.13) e com os parâmetros finais do estudo — os aprovados pelo portão de aceitação da Seção 4.4, registrados em `experiments/config/study_params.json` —, executamos as 56 instâncias com 30 sementes por método estocástico. A Tabela 6 resume o desempenho global e a Tabela 7 o detalha por família; os resultados de cada uma das 56 instâncias estão no Apêndice A.

Tabela 6 – Desempenho global nas 56 instâncias (gap de distância ao melhor conhecido; 30 execuções por instância para GRASP e GRASP reativo, uma para os determinísticos). *Iterações* e *tempo* são médias por execução; a iteração tem custo diferente em cada método, e os dois números tornam esse esforço explícito.

Algoritmo	Gap médio (%)	Gap melhor (%)	Veículos	Iterações	Tempo (s)
Solomon I1	35,97	—	8,46	—	0,000
VND	8,43	—	8,36	—	0,037
GRASP	1,96	0,99	8,30	1249	57,2
GRASP reativo	1,88	1,02	8,29	1249	56,6
Busca Tabu	3,95	—	8,29	1088	7,6

Como ler a Tabela 6. O *gap médio* agrega, sobre as 56 instâncias, a média das 30 execuções de cada método estocástico; o *gap melhor* toma a melhor execução por instância — a distância entre os dois mede o quanto a aleatoriedade ajuda. As colunas de iterações e tempo existem porque o critério de parada é o mesmo, mas o custo de uma iteração não: a tabela mostra o GRASP gastando, em média, cerca de 57 s por execução contra 7,6 s da Busca Tabu (médias sobre as 56 instâncias, medidas sob a execução paralela do *pipeline* — por isso maiores que as medianas sem concorrência da Tabela 3). Essa razão, porém, depende da instância — varia de $0,6\times$ (Tabu mais lenta) a $17\times$ (mediana $7,4\times$) — de modo que “a Tabu responde mais rápido” é uma caracterização *média*, não uma propriedade garantida por instância.

A Tabela 6 sustenta três leituras. A cadeia construção $\rightarrow$ busca local $\rightarrow$ meta-heurística rende, em média, a queda de 35,97% (I1 pura) para 8,43% (VND) e daí para menos de 2% (GRASP e Reativo): cada camada do estudo contribui, e a maior contribuição individual é da busca local. GRASP e Reativo terminam praticamente empatados (1,96% e 1,88%), com a Busca Tabu atrás (3,95%); o parágrafo de significância, adiante, testa se essas diferenças são estatisticamente reais. Por fim, o número médio de veículos dos métodos ($\approx 8,3$) fica entre o das referências de mínima distância e o das de mínima frota, como esperado sob o objetivo de distância (Seção 4.7).

Tabela 7 – Gap médio de distância por família (%). As famílias diferem na geografia dos clientes (C agrupada, R aleatória, RC mista), e a dificuldade relativa dos métodos muda com ela.

Algoritmo	C (agrupada)	R (aleatória)	RC (mista)
Solomon I1	26,58	37,28	44,05
VND	3,36	10,98	10,14
GRASP	0,14	2,54	3,05
GRASP reativo	0,09	2,59	2,77
Busca Tabu	0,61	4,89	6,14

Por família (Tabela 7), o padrão é consistente: na família C, de clientes agrupados, todos os métodos de melhoria chegam perto do ótimo — a estrutura de *clusters* deixa pouco espaço para reotimização — enquanto R e RC, de geografia dispersa, separam os métodos com clareza. É nelas que a diferença entre as meta-heurísticas e o VND se acentua.

Significância estatística. O resultado primário usa o **conjunto de teste** (28 instâncias que a calibração nunca viu), imune a superajuste. O teste de Friedman rejeita a igualdade entre os cinco métodos com folga ($\chi_F^2 = 94,15$, $p = 1,7 \times 10^{-19}$), com ranks médios GRASP 1,77, Reativo 1,84, Tabu 2,63, VND 3,77 e I1 5,00. As comparações par a par (Wilcoxon com Holm, tabela completa em `results/stats_paired.txt`) dão o quadro final: **todos os pares diferem significativamente, exceto GRASP $\times$ Reativo** ($p = 0,95$). Em particular, GRASP e Reativo superam a Busca Tabu ($p_{\text{Holm}} = 6,9 \times 10^{-4}$, com o GRASP melhor em 18 das 28 instâncias e pior em 3). A hierarquia empírica é, portanto, $\{\text{GRASP} \approx \text{Reativo}\} \succ \text{Tabu} \succ \text{VND} \succ \text{I1}$, sintetizada no diagrama de diferença crítica da Figura 5. Nas 56 instâncias o quadro é o mesmo ($\chi_F^2 = 185,36$, $p = 5,3 \times 10^{-39}$). A conclusão é robusta à limitação do pós-teste por ranks discutida na Seção 3.14: nas 56 instâncias, a diferença de rank GRASP–Tabu (0,83) excede a diferença crítica tanto com a I1 no conjunto comparado ($CD = 0,815$) quanto sem ela ($CD = 0,627$).

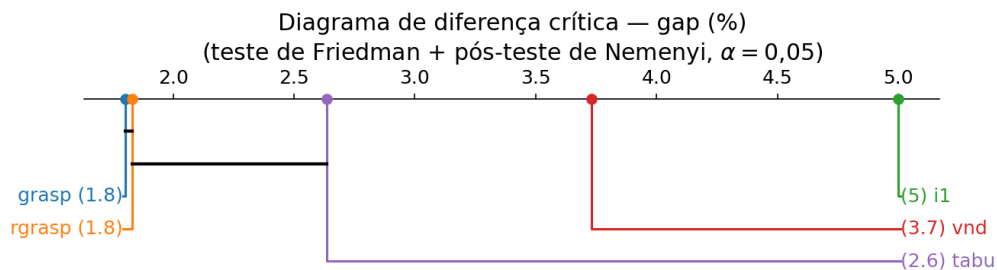


Figura 5 – Diagrama de diferença crítica (Friedman/Nemenyi, 56 instâncias). Cada método aparece na posição do seu rank médio (esquerda = melhor); uma barra horizontal une métodos cuja diferença de ranks é menor que a diferença crítica CD — estatisticamente indistinguíveis a 5%. A barra diz quem *não* se separa; a ausência de barra entre dois métodos indica que a diferença observada dificilmente é acaso.

Convergência e tempo-alvo (TTT). A Figura 6 mostra a queda do custo ao longo do tempo na instância R101 — cada curva é o melhor valor corrente de um método, e o formato “escada” reflete que o incumbente só muda quando uma solução melhor é encontrada. A análise tempo-alvo (Aiex et al., 2007) (Figura 7) fixa um alvo de qualidade (1% acima do melhor conhecido) e pergunta: *quanto tempo o método leva para*

alcançá-lo? Cada execução independente fornece um tempo; o gráfico é a distribuição acumulada empírica desses tempos — curvas mais à esquerda indicam método mais rápido, e a inclinação indica a variabilidade entre execuções. A análise TTT usa 100 re-execuções dedicadas do GRASP em R101 (além das 30 do estudo principal, pois a curva empírica pede uma amostra maior); todas as 100 atingiram o alvo.

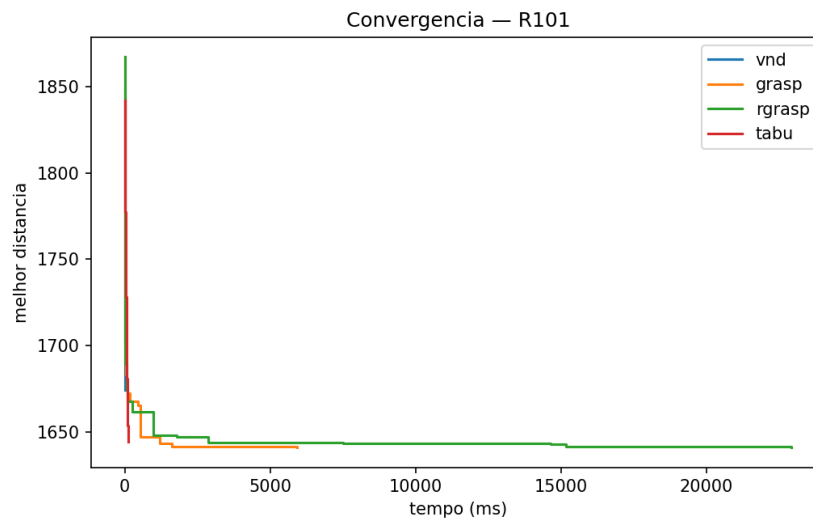


Figura 6 – Convergência do custo (distância) ao longo do tempo na instância R101.

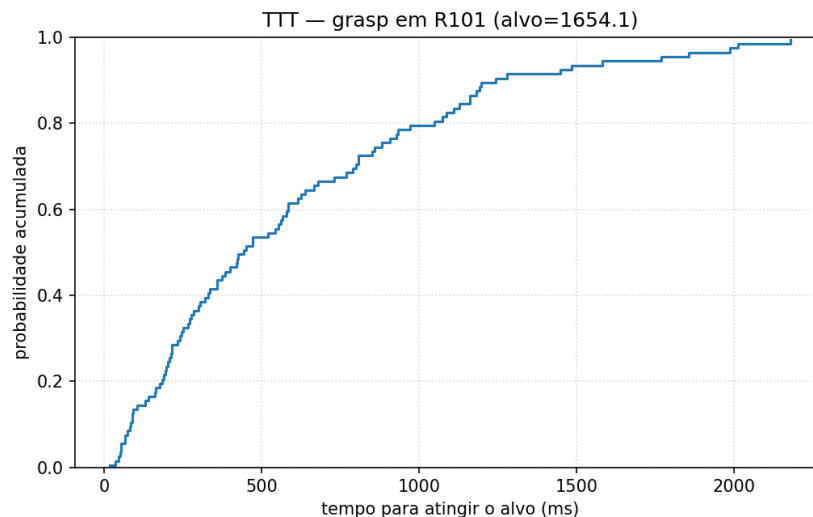


Figura 7 – Tempo-alvo (TTT) do GRASP em R101: probabilidade acumulada de atingir o alvo (custo até 1% acima do melhor conhecido) em função do tempo; 100 execuções.

4.6 Análise complementar: integral primal (DIMACS)

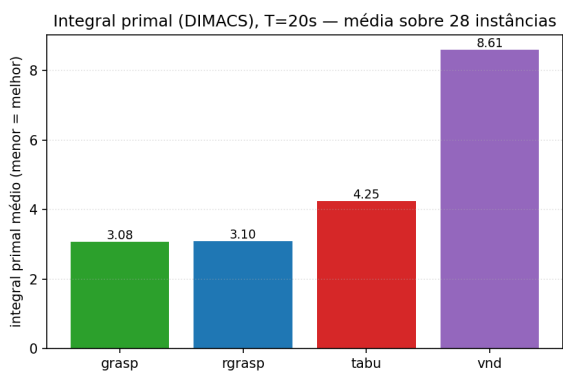
A comparação primária deste estudo usa parada por iterações sem melhora (Seção 3.12), por reprodutibilidade. Como *complemento*, avaliamos a dimensão *anytime* (tempo $\times$

qualidade) pela métrica oficial da *competição* DIMACS, a **integral primal** (Berthold, 2013; DIMACS, 2022): para um orçamento de tempo T e o melhor conhecido BKS ,

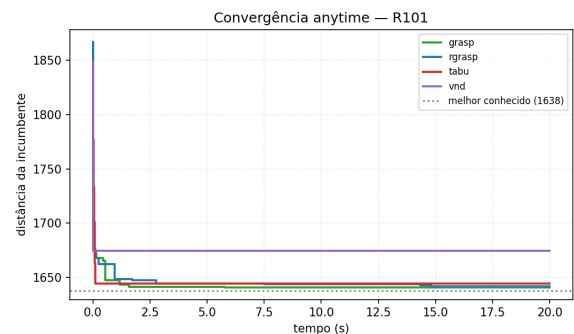
$$PI = 100 \left(\frac{\sum_i v(i-1) (t_i - t_{i-1}) + v(n) (T - t_n)}{T \cdot BKS} - 1 \right),$$

em que $v(0) = 1,1 BKS$ em $t_0 = 0$ (convenção do desafio: antes da primeira solução, o método é tratado como se estivesse a 10% do melhor conhecido, limitando a penalidade inicial) e $v(i)$ é o valor do incumbente encontrado em t_i ; menor PI é melhor (encontra boas soluções mais cedo). Reexecutamos os métodos com orçamento de tempo *fixo* ($T = 20$ s) e os parâmetros de qualidade calibrados. Por depender do tempo de relógio, a integral primal é *dependente da máquina*; por isso é reportada *apenas como complemento* ao estudo reprodutível, e não substitui a comparação primária por iterações.

A Figura 8a resume a integral primal média e a Figura 8b mostra a convergência anytime em R101. A integral primal média ordena os métodos assim: GRASP ($PI = 3,08$), GRASP reativo ($PI = 3,10$), Busca Tabu ($PI = 4,25$), VND ($PI = 8,61$) (menor é melhor). O VND, embora convirja quase instantaneamente, estaciona em seu ótimo local, o que penaliza sua integral primal; os métodos que continuam melhorando ao longo do tempo obtêm os menores valores. Sob tempo igual a leitura é a mesma da comparação primária, com as distâncias entre os métodos encurtadas — coerente com o fato de a Busca Tabu realizar muito mais iterações por segundo.



(a) Integral primal média (menor = melhor).



(b) Convergência anytime em R101.

Figura 8 – Análise complementar anytime (integral primal do DIMACS, $T = 20$ s): métrica dependente de máquina, reportada apenas como complemento.

4.7 O baseline e a contagem de veículos

O *gap* é medido contra um conjunto de soluções de referência de **mínima distância** (convenção DIMACS), coerente com o objetivo otimizado, e disponíveis no repositório CVRPLIB (CVRPLIB, 2026) — repositório oficial do desafio. Coexistem na literatura *três* convenções de arredondamento incompatíveis: a do DIMACS (truncamento a 1 casa decimal, adotada aqui), a do SINTEF (precisão dupla, 2 casas) e a euclidiana real (sem arredondamento); por isso, todos os custos — nossos e das referências — são (re)computados sob a *mesma* convenção DIMACS, garantindo comparabilidade.

Verificamos que essas referências têm distância menor ou igual à das soluções de *mínimo número de veículos* do repositório SINTEF (SINTEF, 2026) em todas as 56 instâncias (em 37, estritamente menor; nas demais, igual), ao custo de utilizarem *mais* veículos. Por exemplo, em R201 a referência de distância usa 8 veículos com custo 1143,2, enquanto a de mínimo de veículos usa 4 veículos com custo 1248,4. Portanto, comparar a distância de nossas soluções com esse baseline é coerente, e a contagem “elevada” de veículos é a assinatura esperada da minimização de distância, não um defeito. A Tabela 8 reporta a visão lexicográfica (veículos e distância) por família, frente às duas referências, e mostra que nossas soluções superam o SINTEF em distância média usando mais veículos, aproximando-se da referência de mínima distância; coerentemente, nosso número médio de veículos situa-se entre o das duas referências.

Tabela 8 – Visão lexicográfica por família: veículos e distância médios das nossas soluções (GRASP, melhor de 30 execuções) e das duas referências — a de mínima distância (CVRPLIB, convenção DIMACS) e a de mínimo número de veículos (SINTEF).

Família	Nosso (GRASP)		CVRPLIB (mín. dist.)		SINTEF (mín. veíc.)	
	veíc.	dist.	veíc.	dist.	veíc.	dist.
C (agrupada)	6,7	714,3	6,7	714,1	6,7	714,1
R (aleatória)	9,0	1042,7	9,5	1029,6	7,5	1081,9
RC (mista)	9,1	1184,2	9,4	1167,6	7,4	1248,3
Todas	8,3	983,4	8,6	973,2	7,2	1017,8

5 Aplicação: Digital Twin para coleta de resíduos

Como terceira camada do estudo, aplicamos os algoritmos *calibrados no benchmark* a um cenário de coleta de resíduos por meio de um *Digital Twin* (gêmeo digital): uma representação virtual de um sistema físico, mantida atualizada por leituras de sensores, na qual decisões de operação podem ser ensaiadas e medidas antes de ir às ruas (Barth et al., 2023) — aqui, o sistema é a rede de coleta seletiva de Vitória/ES, os

sensores informam o enchimento das lixeiras e a decisão ensaiada é o roteamento dos caminhões. A transferência não exigiu infraestrutura nova: o solver é o mesmo binário do *benchmark* (Seção 3.1), que passa a receber matrizes explícitas de distância e tempo (Seção 5.2); a cada ciclo de monitoramento, as lixeiras críticas tornam-se uma instância VRPTW e são roteadas com os parâmetros aprovados na Seção 4.4. Trata-se de uma **prova de conceito**: as localizações são reais, mas o comportamento de enchimento — e, portanto, o “sistema físico” que o gêmeo espelha — é simulado.

Transferibilidade dos parâmetros calibrados. Os parâmetros aplicados aqui são os do estudo estático (Seção 3.13), calibrados nas instâncias euclidianas de Solomon; é preciso justificar por que servem a um cenário de malha viária real e demanda estocástica. O primeiro argumento está no objeto da calibração: o que ela ajusta são parâmetros da *estratégia de busca* (a largura da lista restrita α , a memória de proibição *tenure*, a cadência do aprendizado reativo, dada por δ e *block_frac*), e nada que dependa da geometria da instância — o solver recebe as matrizes viárias explícitas, e os operadores e o teste de viabilidade são exatamente os do *benchmark*. O segundo argumento vem dos próprios resultados do estudo, que limitam o risco prático da transferência: a calibração mostrou-se pouco sensível na faixa útil (o GRASP é insensível a α entre 0,2 e 0,3; Seção 4.4) e, nas instâncias pequenas de cada ciclo, as diferenças entre os métodos de melhoria quase desaparecem. Fica registrada a ressalva: *não* recalibramos no cenário viário, porque isso exigiria um conjunto de treino de cenários simulados e ajustaria os parâmetros ao próprio simulador, não à realidade. Os valores usados devem ser lidos, portanto, como *razoáveis herdados do benchmark*, sem pretensão de serem ótimos no novo cenário; também por isso esta camada se apresenta como prova de conceito.

Dados e enquadramento. Usamos as localizações reais de **38 pontos de entrega voluntária (PEVs)** de coleta seletiva de Vitória/ES, obtidas do serviço de dados abertos do município, com o depósito na associação de triagem AMARIV. Cada PEV corresponde, no modelo, a uma lixeira monitorada — o “cliente” do VRPTW da Seção 2. O *nível de enchimento* de cada lixeira é gerado por um **processo de Poisson não homogêneo** (NHPP) discretizado em ciclos (8 ciclos/dia $\approx$ 3 h cada). Em cada ciclo c , a intensidade é

$$\lambda(c) = \lambda_{\text{off}} + (\lambda_{\text{pico}} - \lambda_{\text{off}}) \min(1, \rho(c)), \quad \rho(c) = e^{-(h-1,5)^2} + e^{-(h-5,5)^2}, \quad h = c \bmod 8, \quad (12)$$

com $\lambda_{\text{off}} = 0,8$ e $\lambda_{\text{pico}} = 4,0$ (dois picos diários). O número de chegadas na lixeira i no ciclo é $\text{Poisson}(\lambda(c) p_i)$, com propensão heterogênea $p_i \sim \mathcal{U}(0,5; 1,5)$, e o enchimento sobe $\Delta f = \text{chegadas}/12$. Uma lixeira *transborda* se f atinge 1 antes de ser coletada — é o KPI de falha de serviço, contado por *episódio*: cada enchimento completo conta um único transbordo, e a lixeira só volta a poder transbordar depois de coletada. Registre-se que *o enchimento é sintético*, calibrado pela literatura de resíduos urbanos (Lozano et al., 2018; Barth et al., 2023), pois a coleta de séries reais exigiria meses de monitoramento — fora do escopo de uma Iniciação Científica. A validação com dados reais de enchimento é trabalho futuro.

5.1 Camada de comunicação LoRaWAN (simulada)

Entre os sensores e o orquestrador há uma camada de comunicação que simula os efeitos relevantes de uma rede LoRaWAN na escala de um ciclo de monitoramento. Sensores de lixeira alimentados por bateria operam tipicamente na classe A do LoRaWAN — o dispositivo passa a maior parte do tempo dormindo e acorda apenas para transmitir (Adelantado et al., 2017; Ramson et al., 2022) — com *uplinks* não confirmados (sem retransmissão); três efeitos dessa escolha são modelados. A **perda de pacote**: cada uplink é entregue com probabilidade *pdr* (*packet delivery ratio*, padrão 0,98), e um pacote perdido não é retransmitido. O **ciclo de trabalho** (*duty cycle*): a banda ISM limita o tempo de ar de cada dispositivo (cerca de 1% na faixa EU868 (Adelantado et al., 2017)), o que na prática limita a frequência de transmissão — cada sensor transmite a cada *duty* ciclos, com fases sorteadas para que a rede não transmita em uníssono. A **defasagem**: o atraso fim a fim de um uplink (segundos) é desprezível frente a um ciclo (horas), de modo que o efeito real da latência nessa escala é a *idade da leitura* — entre transmissões, ou após uma perda, o orquestrador segue operando com a última leitura recebida.

A consequência é uma distinção que o gêmeo digital passa a representar: o roteador não decide sobre o estado físico das lixeiras, e sim sobre a *visão de rádio* desse estado. O acionamento por limiar, a demanda e a janela dinâmica usam o valor percebido; o transbordo continua contado no estado físico, que é o que ocorre na rua. Uma lixeira que cruza o limiar, mas perde o uplink, fica invisível ao planejamento até o próximo uplink entregue, e o transbordo que ocorrer nesse intervalo é atribuível à comunicação — um custo operacional que o modelo sem camada de rádio não capturava. Note-se que o roteador não *reage* a falhas de rádio: como os uplinks são não confirmados, uma perda não gera nenhum aviso, e o orquestrador segue, sem o saber, com a leitura antiga; a única atualização que não depende do rádio é a própria coleta —

ao esvaziar a lixeira, o caminhão observa o estado em campo, e a leitura percebida é zerada nesse instante. Com canal perfeito ($pdr = 1$, transmissão a cada ciclo) a camada é transparente: a visão de rádio coincide com o estado físico em todo ciclo, propriedade verificada por teste automatizado. O efeito do canal é mensurável (artefato em `results/digital_twin/lorawan_channel.csv`): no mapa R101 com 12 ciclos e o VND como roteador, degradar o canal do perfeito ($pdr = 1$, transmissão a cada ciclo) para $pdr = 0,9$ com transmissão a cada 2 ciclos elevou os transbordos de 51 para 96 episódios, com defasagem média de 0,52 ciclo: além do roteamento, a própria qualidade da comunicação limita o serviço.

5.2 Malha viária real (distâncias e tempos de direção)

Para realismo, além do grafo euclidiano (linha reta entre pontos), construímos um **grafo viário real**: a matriz de *distância* (metros) e a matriz de *tempo* (segundos de direção) entre o depósito e os 38 PEVs são obtidas da malha do OpenStreetMap pelo serviço de roteamento OSRM (Luxen e Vetter, 2011), e a geometria de 5 937 ruas é usada para visualização. O fator de circuito médio (distância rodoviária / linha reta) foi de **1,53** (até 6,9 em pares separados por água, já que Vitória é uma ilha). O solver foi estendido para aceitar matrizes *explícitas* de distância e tempo (o *benchmark* euclidiano de Solomon permanece inalterado, pois suas execuções não usam matrizes): assim o custo otimizado passa a ser a distância *rodoviária* e as janelas/espera usam o tempo *real* de direção. O cenário rodoviário é internamente consistente em *unidades físicas* — distância em metros, e tempo, janelas e serviço em segundos ($H = 6$ h, serviço = 120 s) —, independentes das unidades abstratas do *benchmark*. A Figura 9 mostra as rotas resultantes seguindo as ruas reais; rotear os 38 PEVs produziu 6 veículos, 83,7 km de percurso viário e 3,7 h de operação.

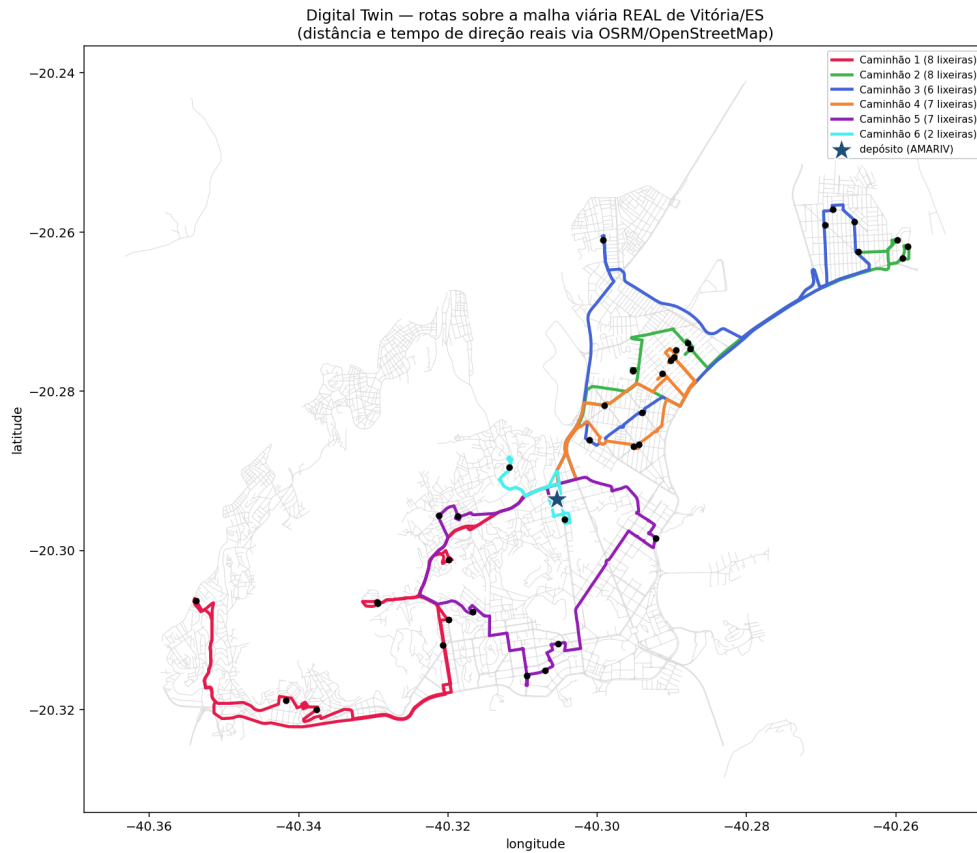


Figura 9 – Rotas do Digital Twin sobre a malha viária *real* de Vitória/ES (OSM): cada cor é um caminhão, traçado pela geometria de direção real (OSRM); a estrela é o depósito (AMARIV).

5.3 Modelo do cenário: capacidades, tempo e espera

Para que não haja qualquer ambiguidade, definimos formalmente cada elemento do cenário de coleta.

Capacidade do veículo (Q). Cada caminhão carrega no máximo Q unidades de demanda. Ao longo de uma rota, a carga acumulada é a soma das demandas das lixeiras visitadas; *nenhuma* rota pode exceder Q . Quando incluir mais uma lixeira excederia Q , a rota é fechada (o caminhão retorna ao depósito) e outro veículo é necessário — é isso que, junto com as janelas, determina o *número de veículos*.

Capacidade da lixeira e demanda. O estado de cada lixeira é o nível de enchimento $f \in [0, 1]$ (0 = vazia, 1 = cheia/transbordando). O enchimento sobe pelo processo de Poisson não homogêneo; ao atingir 1 , a lixeira *transborda* (indicador de falha de serviço). A demanda de coleta é derivada do enchimento por demanda = $\text{round}(9f) + 1$ (arredondamento ao inteiro mais próximo; entre 1 e 10): é uma medida de *prioridade/volume a recolher*, não o peso físico — lixeiras mais cheias recebem demanda maior e ocupam mais da capacidade do veículo.

Tempo, chegada e espera. O tempo de viagem entre dois pontos é a *duração real de direção* pela malha viária (segundos, OSRM); no grafo euclidiano, é proporcional à distância (velocidade constante). Partindo do depósito em $t = 0$, a cada perna da rota: chegada = partida_{anterior} + tempo de viagem. Se a chegada ocorre *antes* da abertura da janela da lixeira (*ready*), o veículo **espera, ocioso**, até *ready*: o início de serviço é início = max(chegada, *ready*). O serviço dura *service* segundos e partida = início + *service*. A rota deve retornar ao depósito até o fim do horizonte (*due* do depósito). Uma janela é *violada* (solução inviável) se início > *due* da lixeira. Esse é exatamente o mecanismo ilustrado numericamente no exemplo da Seção 3.10(a).

Janelas de tempo dinâmicas. A cada ciclo de monitoramento, as lixeiras acima de um limiar de enchimento τ (padrão 0,7) tornam-se demandas de coleta com **janela de tempo dinâmica** $[0, due]$: quanto mais cheia (mais crítica) a lixeira, mais cedo seu prazo. Formalmente, com criticidade $crit = \max(0, (f - \tau)/(1 - \tau)) \in [0, 1]$,

$$due = H (1 - \kappa \cdot crit), \quad (13)$$

onde H é o horizonte e κ (padrão 0,6) é o *coeficiente de urgência* — analisado na sensibilidade (Figura 10). O mecanismo age pela folga Push Forward do solver (Seção 2.3): prazos mais curtos restringem as posições de inserção viáveis e tendem a antecipar as lixeiras críticas dentro das rotas. Se essa priorização chega a alterar os indicadores agregados é uma pergunta empírica, respondida para esta configuração pela análise de sensibilidade (Figura 10). Constrói-se então uma instância VRPTW e invoca-se o solver (com os parâmetros calibrados e o mesmo critério de parada do estudo) para (re)planejar as rotas; as lixeiras atendidas são esvaziadas.

Uma simplificação do modelo precisa ser explicitada, porque ela responde a uma pergunta natural: se o ciclo de monitoramento dura cerca de 3 h e o horizonte de planejamento pode chegar a 6 h no cenário viário, o que acontece com uma coleta agendada para a 5ª hora quando o ciclo seguinte replaneja tudo? No gêmeo digital, nada fica pendente: o plano de cada ciclo é *atômico* — as rotas planejadas são executadas por inteiro dentro do próprio ciclo, e todas as lixeiras roteadas são esvaziadas antes de o ciclo seguinte começar. O horizonte H e as janelas dinâmicas atuam, assim, como mecanismo de *priorização dentro do plano* (quais lixeiras são atendidas mais cedo em cada rota), e não como um calendário que atravessa ciclos; não há sobreposição de estado entre um plano e o seguinte. A simplificação está na mesma granularidade temporal da camada LoRaWAN (Seção 5.1); relaxá-la, com planos que atravessam ciclos e replanejamento de veículos em rota, fica como trabalho futuro.

O que influencia o número de veículos e o esvaziamento das rotas. Três fatores governam a operação simulada e explicam o comportamento do sistema: (i) o *limiar de acionamento* — limiares menores acionam mais lixeiras por ciclo, exigindo mais veículos; (ii) a *capacidade do veículo* frente à *demanda* de cada lixeira (que cresce com o enchimento, como heurística de priorização) — quanto menor a razão capacidade/demanda, mais rotas; e (iii) as *janelas dinâmicas* — prazos mais apertados reduzem o compartilhamento de rotas e podem aumentar o número de veículos, efeito que, no horizonte folgado da configuração avaliada, não chegou a se manifestar (Figura 10). Uma rota “esvazia” (e um veículo é liberado) quando os movimentos entre rotas conseguem redistribuir todos os seus clientes sem violar viabilidade.

Validação: regime dinâmico vs. estático. Comparam-se dois regimes de operação: o **dinâmico** (coleta sob demanda das lixeiras acima do limiar a cada ciclo) e o **estático** (coleta de todas as lixeiras a cada ϕ ciclos, independentemente do enchimento). A comparação é *pareada* sobre 30 realizações independentes de enchimento: em cada semente, os dois regimes veem exatamente a mesma sequência de chegadas, e as diferenças por semente alimentam um teste de Wilcoxon pareado, no espírito do protocolo estatístico do estudo estático. Para as diferenças nulas, frequentes em indicadores inteiros, usa-se aqui o tratamento *zsplit*, que divide os empates igualmente entre os postos positivos e negativos em vez de descartá-los como na Seção 3.14 — descartar os muitos zeros esvaziaria a amostra. Os indicadores são distância total, número de coletas e **transbordos** (por episódio). Como os parâmetros operacionais (limiar, capacidade, frequência ϕ , coeficiente de urgência) são escolhas de modelagem, reportamos uma **análise de sensibilidade** variando o limiar, a capacidade e o coeficiente de urgência (a frequência do regime estático foi mantida em $\phi = 3$ ciclos), em vez de fixá-los arbitrariamente; e não estendemos as conclusões além do que a simulação suporta. Um painel interativo (*dashboard*) permite reproduzir e visualizar os ciclos.

Resultados (prova de conceito em Vitória/ES). Sobre simulações de 16 ciclos em 30 realizações independentes de enchimento, no grafo euclidiano dos 38 PEVs e com o canal LoRaWAN padrão ($pdr = 0,98$), o quadro é de *compromisso*, não de dominância (Tabela 9). O regime **dinâmico** realiza *pouco mais da metade das coletas* do **estático** (124 contra 228, em média — uma redução de 46% do esforço), que é a economia operacional que motiva a coleta guiada por sensores. Em contrapartida, no limiar de acionamento padrão ($\tau = 0,7$), ele incorre em *mais transbordos* (27,0 contra 19,1 episódios, $p < 0,001$) e distância comparável, ligeiramente maior ($p = 0,0016$): esperar a lix-

eira encher 70% para agendá-la deixa pouca margem antes do transbordo. A análise de sensibilidade adiante indica que esse compromisso é governado pelo próprio limiar: com $\tau = 0,5$, os transbordos da realização analisada caem para 5, ainda com menos coletas que o estático. Parte dos transbordos do regime dinâmico é atribuível à comunicação, não ao roteamento: com o canal degradado da Seção 5.1, essa parcela cresce. Em suma, *a coleta dinâmica compra grande economia de coletas, e o nível de serviço resultante depende de onde se coloca o limiar.*

Aplicando os cinco algoritmos à mesma simulação, os métodos de melhoria convergem a rotas de custo muito próximo: nas instâncias *pequenas* de cada ciclo (tipicamente 10–20 lixeiras ativas), a diferença entre as meta-heurísticas quase desaparece — as distinções medidas no *benchmark* de 100 clientes manifestam-se em problemas maiores.

Tabela 9 – Gêmeo digital (Vitória/ES, 16 ciclos, canal LoRaWAN com $pdr = 0,98$): coleta dinâmica guiada pelos sensores contra coleta estática de frequência fixa (a cada 3 ciclos), sobre 30 realizações independentes de enchimento — os dois regimes veem a *mesma* realização em cada semente (comparação pareada). Células: média $\pm$ IC95%. Transbordos contados por *episódio* (a lixeira atinge a capacidade antes da coleta; uma contagem por enchimento). Última linha: p do teste de Wilcoxon pareado sobre as 30 sementes.

Regime	Distância	Coletas	Transbordos	Ciclos com rota
Dinâmico (sob demanda)	3751,0 $\pm$ 72,7	123,9 $\pm$ 2,5	27,0 $\pm$ 2,0	15,0 $\pm$ 0,1
Estático (a cada 3 ciclos)	3663,6 $\pm$ 36,9	228,0 $\pm$ 0,0	19,1 $\pm$ 2,1	6,0 $\pm$ 0,0
p (Wilcoxon pareado)	0,0016	0,0000	0,0000	0,0000

Sensibilidade dos parâmetros. A Figura 10 varia um parâmetro por vez, sobre uma mesma realização de enchimento, e mostra o efeito de cada um. O *limiar de coleta* domina o compromisso serviço–custo: limiares mais altos reduzem coletas, mas aumentam transbordos (de 5 a 61 episódios ao variar τ de 0,5 a 0,9). A *capacidade* do veículo reduz fortemente a distância (de 6527 a 3049 ao variar de 15 a 40), sem afetar transbordos. O *coeficiente de urgência*, por sua vez, não teve efeito mensurável nos indicadores agregados desta configuração: com o horizonte folgado do cenário euclidiano (nas unidades abstratas do *benchmark*, em contraste com as unidades físicas da Seção 5.2), as janelas dinâmicas raramente chegam a restringir a viabilidade — elas atuam apenas na priorização interna ao plano, coerentemente com a simplificação de atomicidade discutida acima, no parágrafo das janelas dinâmicas. Esses resultados delimitam o papel de cada parâmetro e apontam o limiar como a alavanca operacional decisiva.

Digital Twin (Vitória/ES, prova de conceito) — compromisso distância x transbordos [grasp]

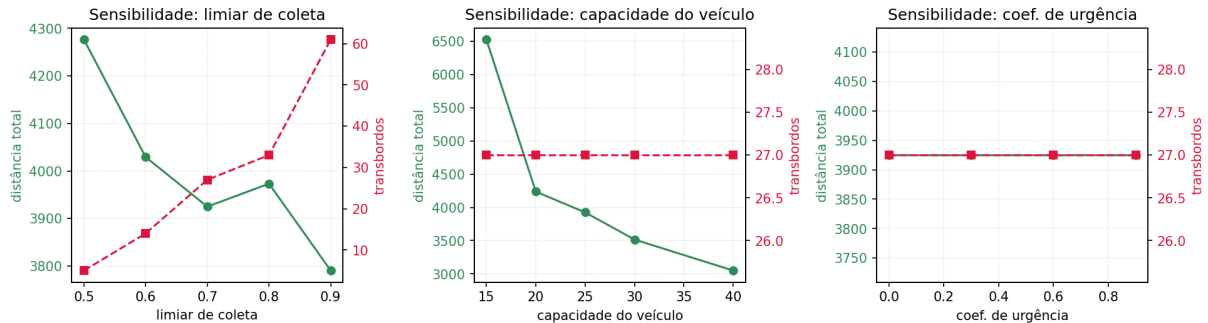


Figura 10 – Análise de sensibilidade do Digital Twin: distância (verde) e transbordos (vermelho) ao variar limiar, capacidade e coeficiente de urgência. O compromisso serviço–custo aparece apenas no limiar; a capacidade reduz a distância sem afetar transbordos; o coeficiente de urgência não altera os indicadores agregados.

6 Atividades realizadas

As Etapas 1 a 3 (revisão bibliográfica, aprofundamento no algoritmo de Solomon e nas tecnologias de sensores, e definição da arquitetura) foram concluídas. A Etapa 4 (implementação dos algoritmos e do ambiente de simulação) foi concluída: o solver com os cinco métodos sobre o núcleo compartilhado está implementado, testado e verificado. A Etapa 5 (relatório parcial) foi entregue. As Etapas 6 e 7 (Digital Twin e testes computacionais) foram concluídas: a calibração por *irace*, a comparação estatística nas 56 instâncias e o Digital Twin com dados reais de Vitória estão operacionais e produziram os resultados das Seções 4 e 5. As Etapas 8 e 9 (análise final e relatório) encerram-se com este documento. O Quadro 1 apresenta o cronograma completo.

Quadro 1 – Cronograma de atividades previstas e realizadas do Subprojeto (set./2025 a ago./2026)

Atividade	set.	out.	nov.	dez.	jan.	fev.	mar.	abr.	mai.	jun.	jul.	ago.
Etapa 1 – Estudo dos conceitos básicos, modelagem estatística e revisão bibliográfica sobre otimização de rotas e Digital Twins	✓	✓										
Etapa 2 – Aprofundamento teórico no algoritmo de Solomon e nas tecnologias de sensores IoT aplicáveis		✓	✓	✓								
Etapa 3 – Proposta de adaptação do algoritmo de Solomon e definição da arquitetura do sistema				✓	✓							
Etapa 4 – Desenvolvimento dos módulos de simulação, integração de sensores IoT e testes iniciais					✓	✓	✓	✓				
Etapa 5 – Elaboração do Relatório Parcial da Pesquisa						✓	✓					
Etapa 6 – Desenvolvimento da plataforma Digital Twin e da interface de visualização de dados								✓	✓	✓		
Etapa 7 – Testes computacionais com o algoritmo e coleta/integração de dados reais									✓	✓	✓	
Etapa 8 – Análise dos resultados e validação do modelo no sistema										✓	✓	✓
Etapa 9 – Elaboração do Relatório Final e documentação do projeto											✓	✓

Fonte: Produção da própria autora.

Nota: atividade totalmente realizada (✓), parcialmente realizada (◐) e não realizada (○).

Descrição do Quadro 1: cronograma de 9 etapas ao longo de 12 meses (setembro de 2025 a agosto de 2026). Todas as etapas foram concluídas; em relação ao relatório parcial, as Etapas 6 a 9, antes previstas como futuras, foram realizadas entre abril e agosto de 2026.

7 Conclusão

Este trabalho construiu uma comparação controlada de cinco heurísticas clássicas para o VRPTW: núcleo compartilhado, viabilidade garantida por construção e conferida por um verificador independente, parada por iterações sem melhora (que independe da máquina) e comparação estatística por Friedman/Nemenyi. As escolhas metodológicas de objetivo, baseline, conjunto de vizinhanças e calibração foram justificadas uma a uma, com apoio empírico da ablação e da verificação contra referências publicadas.

Quantitativamente, a hierarquia empírica é $\{\text{GRASP} \approx \text{GRASP reativo}\} \succ \text{Busca Tabu} \succ \text{VND} \succ \text{I1}$: cada camada de refinamento reduz o gap médio de distância — de 35,97% (construção I1 pura) para 8,43% (VND) e para menos de 2% nas duas variantes de GRASP, com a Busca Tabu em posição intermediária (3,95%). Todas as separações são estatisticamente significativas no conjunto de teste (Wilcoxon com Holm), exceto entre o GRASP fixo e o reativo, indistinguíveis. Sobre a calibração, o resultado varia por método: houve ganho demonstrável para a Busca Tabu (+1,26 pp), ganho marginal para o Reativo e nenhum para o GRASP — que se mostrou insensível a α na faixa estudada e, pelo protocolo pré-declarado, usa o valor clássico no estudo final. A verificação reproduziu os 56 custos de referência e *nenhuma* das 3 528 execuções do estudo produziu solução inviável, confirmando que os algoritmos de melhoria respeitam cobertura, capacidade e janelas de tempo a cada passo. Transportados ao Digital Twin de coleta em Vitória/ES, agora incluindo a camada de comunicação LoRaWAN simulada, os algoritmos mostraram um compromisso operacional claro. A coleta dinâmica guiada por sensores realiza pouco mais da metade das coletas do regime de frequência fixa, e o nível de serviço é governado pelo limiar de acionamento: no limiar padrão, os transbordos superam os do regime fixo; limiares mais baixos recuperam o serviço mantendo a economia de coletas. Além do roteamento, a própria qualidade do canal de comunicação limita o serviço.

Limitações. O objetivo otimizado é a distância (DIMACS); a abordagem lexicográfica de Solomon foi reportada apenas de forma complementar. A Busca Tabu foi mantida na forma mais simples (sem diversificação avançada). No Digital Twin, o enchimento das lixeiras é simulado, não medido; a comunicação LoRaWAN é representada, não validada energeticamente; os hiperparâmetros são herdados do *benchmark* euclidiano, sem recalibração no cenário viário; e a análise de sensibilidade usa uma única realização de enchimento (a comparação dinâmico $\times$ estático, por sua vez, usa 30).

Trabalhos futuros. Validação com dados reais de enchimento em Vitória (piloto com sensores); avaliação sob o critério lexicográfico veículos→distância; variantes mais ricas (Tabu com memória de longo prazo, metaheurísticas híbridas/ILS); planos que atravessam ciclos no Digital Twin; e quantificação de benefícios econômicos e ambientais.

Agradecimentos

À Universidade Federal do Espírito Santo e ao Programa Institucional de Iniciação Científica (PIIC/UFES), pelo apoio institucional a esta pesquisa; e à orientadora, pela condução do trabalho.

Referências

- ADELANTADO, F.; VILAJOSANA, X.; TUSET-PEIRO, P.; MARTINEZ, B.; MELIÀ-SEGÚI, J.; WATTEYNE, T. Understanding the limits of LoRaWAN. *IEEE Communications Magazine*, v. 55, n. 9, p. 34–40, 2017.
- ALEX, R. M.; RESENDE, M. G. C.; RIBEIRO, C. C. TTT plots: a perl program to create time-to-target plots. *Optimization Letters*, v. 1, p. 355–366, 2007.
- ALMEIDA, L. G. de; BORIN, J. F. Plataforma inteligente e sustentável para coleta de resíduos baseada em IoT e LPWAN. In: WORKSHOP DE COMPUTAÇÃO URBANA (COURB), 2021. *Anais [...]*. Porto Alegre: SBC, 2021. p. 154–167.
- BARTH, L.; SCHWEIGER, L.; BENEDECH, R.; EHRAT, M. From data to value in smart waste management: optimizing solid waste collection with a digital twin-based decision support system. *Decision Analytics Journal*, v. 9, p. 100347, 2023.
- BENAVOLI, A.; CORANI, G.; MANGILI, F. Should we really use post-hoc tests based on mean-ranks? *Journal of Machine Learning Research*, v. 17, n. 5, p. 1–10, 2016.
- BERTHOLD, T. Measuring the impact of primal heuristics. *Operations Research Letters*, v. 41, n. 6, p. 611–614, 2013.
- CVRPLIB. *Capacitated Vehicle Routing Problem Library — instâncias Solomon do VRPTW e melhores soluções conhecidas*. PUC-Rio, 2026. Disponível em: <https://galgos.inf.puc-rio.br/cvrplib/>. Acesso em: 12 ago. 2026.
- DEMŠAR, J. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, v. 7, p. 1–30, 2006.
- DIMACS. *12th DIMACS Implementation Challenge — Vehicle Routing: VRPTW*

- track (Competition Rules)*. Rutgers University, 2021–2022. Disponível em: <http://dimacs.rutgers.edu/programs/challenge/vrp/vrptw/>. Acesso em: 12 ago. 2026.
- FEO, T. A.; RESENDE, M. G. C. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, v. 6, n. 2, p. 109–133, 1995.
- GARCÍA, S.; HERRERA, F. An extension on “Statistical comparisons of classifiers over multiple data sets” for all pairwise comparisons. *Journal of Machine Learning Research*, v. 9, p. 2677–2694, 2008.
- GLOVER, F. Tabu search: part I. *ORSA Journal on Computing*, v. 1, n. 3, p. 190–206, 1989.
- GLOVER, F. Tabu search: part II. *ORSA Journal on Computing*, v. 2, n. 1, p. 4–32, 1990.
- GLOVER, F.; LAGUNA, M. *Tabu Search*. Boston: Kluwer Academic Publishers, 1997.
- HANSEN, P.; MLADENović, N. Variable neighborhood search: principles and applications. *European Journal of Operational Research*, v. 130, n. 3, p. 449–467, 2001.
- LÓPEZ-IBÁÑEZ, M. et al. The irace package: iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, v. 3, p. 43–58, 2016.
- LOZANO, Á. et al. Smart waste collection system with low consumption LoRaWAN nodes and route optimization. *Sensors*, v. 18, n. 5, p. 1465, 2018.
- LUXEN, D.; VETTER, C. Real-time routing with OpenStreetMap data. In: ACM SIGSPATIAL INTERNATIONAL CONFERENCE ON ADVANCES IN GEOGRAPHIC INFORMATION SYSTEMS, 19., 2011. *Proceedings [...]*. New York: ACM, 2011. p. 513–516.
- OR, I. *Traveling salesman-type combinatorial problems and their relation to the logistics of regional blood banking*. Tese (Doutorado) — Northwestern University, Evanston, 1976.
- PARDINI, K. et al. A smart waste management solution geared towards citizens. *Sensors*, v. 20, n. 8, p. 2380, 2020.
- POTVIN, J.-Y.; ROUSSEAU, J.-M. An exchange heuristic for routing problems with time windows. *Journal of the Operational Research Society*, v. 46, n. 12, p. 1433–1446, 1995.
- PRAIS, M.; RIBEIRO, C. C. Reactive GRASP: an application to a matrix decomposition problem in TDMA traffic assignment. *INFORMS Journal on Computing*, v. 12, n. 3, p. 164–176, 2000.

- RAMSON, S. R. J. et al. A LoRaWAN IoT-enabled trash bin level monitoring system. *IEEE Transactions on Industrial Informatics*, v. 18, n. 2, p. 786–795, 2022.
- RESENDE, M. G. C.; RIBEIRO, C. C. Greedy randomized adaptive search procedures. In: GLOVER, F.; KOCHENBERGER, G. A. (ed.). *Handbook of Metaheuristics*. Boston: Kluwer Academic Publishers, 2003. p. 219–249.
- SINTEF. *Solomon benchmark — 100 customers: best known solutions (minimum vehicles)*. SINTEF Transport Optimisation Portal (TOP), 2026. Disponível em: <https://www.sintef.no/projectweb/top/vrptw/100-customers/>. Acesso em: 12 ago. 2026.
- SOLOMON, M. M. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, v. 35, n. 2, p. 254–265, 1987.
- TAILLARD, É.; BADEAU, P.; GENDREAU, M.; GUERTIN, F.; POTVIN, J.-Y. A tabu search heuristic for the vehicle routing problem with soft time windows. *Transportation Science*, v. 31, n. 2, p. 170–186, 1997.
- TOTH, P.; VIGO, D. (ed.). *The Vehicle Routing Problem*. Philadelphia: SIAM, 2002.

A Resultados detalhados por instância

A Tabela 10 apresenta, para cada uma das 56 instâncias, a comparação dos cinco algoritmos nas métricas: número de veículos, custo (distância média), gap percentual ao melhor conhecido, número de iterações e tempo. Para os algoritmos estocásticos (GRASP e GRASP reativo), iterações e tempo são médias sobre as 30 execuções. O menor gap de cada instância é destacado em negrito.

Tabela 10 – Resultados por instância: comparação dos cinco algoritmos. Custo = distância média; Gap em % ao melhor conhecido; Iter. e Tempo são médias (30 execuções para os estocásticos). O menor gap de cada instância está em negrito.

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
C101	I1	10,0	922,0	11,45	0	0,00
	VND	10,0	827,3	0,00	0	0,01
	GRASP	10,0	827,3	0,00	802	15,74
	GRASP-R	10,0	827,3	0,00	801	15,19
	Tabu	10,0	827,3	0,00	806	8,64
C102	I1	10,0	1039,7	25,67	0	0,00
	VND	10,0	827,3	0,00	0	0,02
	GRASP	10,0	827,3	0,00	806	32,66

continua na próxima página

continuação da Tabela 10

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
	GRASP-R	10,0	827,3	0,00	801	28,76
	Tabu	10,0	827,3	0,00	816	4,90
C103	I1	11,0	1221,7	47,85	0	0,00
	VND	10,0	881,0	6,62	0	0,03
	GRASP	10,0	826,3	0,00	870	40,96
	GRASP-R	10,0	826,3	0,00	811	34,90
	Tabu	10,0	826,3	0,00	1027	5,52
C104	I1	10,0	1148,2	39,53	0	0,00
	VND	10,0	876,7	6,54	0	0,02
	GRASP	10,0	825,2	0,29	1518	65,56
	GRASP-R	10,0	822,9	0,00	857	32,84
	Tabu	10,0	822,9	0,00	1662	9,54
C105	I1	10,0	878,9	6,24	0	0,00
	VND	10,0	827,3	0,00	0	0,01
	GRASP	10,0	827,3	0,00	803	20,51
	GRASP-R	10,0	827,3	0,00	801	17,85
	Tabu	10,0	827,3	0,00	806	8,65
C106	I1	11,0	951,2	14,98	0	0,00
	VND	10,0	827,3	0,00	0	0,01
	GRASP	10,0	827,3	0,00	803	23,95
	GRASP-R	10,0	827,3	0,00	801	20,35
	Tabu	10,0	827,3	0,00	812	4,33
C107	I1	10,0	902,4	9,08	0	0,00
	VND	10,0	827,3	0,00	0	0,01
	GRASP	10,0	827,3	0,00	805	25,12
	GRASP-R	10,0	827,3	0,00	801	19,90
	Tabu	10,0	827,3	0,00	806	8,25
C108	I1	10,0	975,2	17,88	0	0,00
	VND	10,0	827,3	0,00	0	0,01
	GRASP	10,0	827,3	0,00	812	27,47
	GRASP-R	10,0	827,3	0,00	802	22,70
	Tabu	10,0	827,3	0,00	819	4,37
C109	I1	11,0	933,1	12,79	0	0,00
	VND	10,0	847,5	2,44	0	0,01
	GRASP	10,0	827,3	0,00	822	28,07
	GRASP-R	10,0	827,3	0,00	803	23,85
	Tabu	10,0	827,3	0,00	821	4,79
	I1	3,0	609,1	3,40	0	0,00

continua na próxima página

C201

continuação da Tabela 10

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
	VND	3,0	589,1	0,00	0	0,01
	GRASP	3,0	589,1	0,00	801	8,24
	GRASP-R	3,0	589,1	0,00	801	10,80
	Tabu	3,0	589,1	0,00	801	14,42
C202	I1	4,0	793,0	34,61	0	0,00
	VND	3,0	617,9	4,89	0	0,03
	GRASP	3,0	589,1	0,00	802	32,52
	GRASP-R	3,0	589,1	0,00	802	36,21
	Tabu	3,0	617,9	4,89	815	6,68
C203	I1	4,0	931,4	58,21	0	0,00
	VND	4,0	635,3	7,92	0	0,05
	GRASP	3,0	592,0	0,56	1066	61,81
	GRASP-R	3,0	589,0	0,05	1158	71,65
	Tabu	3,0	588,7	0,00	929	8,61
C204	I1	4,0	1138,4	93,57	0	0,00
	VND	4,0	689,7	17,28	0	0,08
	GRASP	3,0	596,9	1,50	1213	83,57
	GRASP-R	3,0	596,7	1,47	1503	108,00
	Tabu	4,0	620,5	5,51	1220	9,62
C205	I1	4,0	667,6	13,85	0	0,00
	VND	3,0	606,8	3,48	0	0,01
	GRASP	3,0	586,4	0,00	802	14,88
	GRASP-R	3,0	586,4	0,00	803	16,18
	Tabu	3,0	586,4	0,00	812	6,70
C206	I1	3,0	660,6	12,73	0	0,00
	VND	3,0	586,0	0,00	0	0,01
	GRASP	3,0	586,0	0,00	801	20,53
	GRASP-R	3,0	586,0	0,00	801	20,50
	Tabu	3,0	586,0	0,00	810	6,66
C207	I1	3,0	747,1	27,54	0	0,00
	VND	3,0	585,8	0,00	0	0,02
	GRASP	3,0	585,8	0,00	801	19,95
	GRASP-R	3,0	585,8	0,00	801	20,60
	Tabu	3,0	585,8	0,00	814	6,67
C208	I1	3,0	717,6	22,50	0	0,00
	VND	3,0	632,3	7,94	0	0,02
	GRASP	3,0	585,8	0,00	801	23,57
	GRASP-R	3,0	585,8	0,00	801	23,50

continua na próxima página

continuação da Tabela 10

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
	Tabu	3,0	585,8	0,00	821	7,37
R101	I1	21,0	1867,1	14,01	0	0,00
	VND	21,0	1674,5	2,25	0	0,02
	GRASP	20,0	1640,6	0,18	1400	26,77
	GRASP-R	20,0	1641,7	0,25	1472	28,45
	Tabu	20,0	1644,3	0,40	828	3,52
R102	I1	19,0	1711,0	16,66	0	0,00
	VND	19,0	1493,0	1,80	0	0,03
	GRASP	18,0	1466,8	0,01	1384	49,13
	GRASP-R	18,0	1467,4	0,05	1379	49,09
	Tabu	19,0	1491,0	1,66	837	3,94
R103	I1	16,0	1550,0	28,24	0	0,00
	VND	16,0	1272,0	5,24	0	0,03
	GRASP	14,2	1221,8	1,08	1406	47,62
	GRASP-R	14,3	1223,1	1,19	1425	48,03
	Tabu	15,0	1240,4	2,62	1803	9,29
R104	I1	12,0	1267,5	30,47	0	0,00
	VND	12,0	1035,8	6,62	0	0,03
	GRASP	11,3	999,7	2,91	1297	44,32
	GRASP-R	11,1	998,7	2,80	1468	50,17
	Tabu	12,0	1022,6	5,26	887	4,41
R105	I1	15,0	1593,4	17,57	0	0,00
	VND	15,0	1414,4	4,36	0	0,02
	GRASP	15,6	1370,0	1,09	1513	37,19
	GRASP-R	15,4	1371,1	1,16	1330	30,97
	Tabu	15,0	1380,6	1,87	844	4,19
R106	I1	14,0	1446,6	17,17	0	0,00
	VND	14,0	1293,2	4,75	0	0,03
	GRASP	13,3	1251,2	1,35	1316	41,72
	GRASP-R	13,1	1252,7	1,46	1481	45,68
	Tabu	14,0	1274,5	3,23	1026	5,21
R107	I1	12,0	1354,1	27,19	0	0,00
	VND	12,0	1159,1	8,88	0	0,02
	GRASP	11,9	1092,0	2,58	1271	40,91
	GRASP-R	11,7	1094,5	2,80	1472	46,55
	Tabu	12,0	1103,3	3,63	1044	5,56
R108	I1	11,0	1226,0	31,53	0	0,00
	VND	11,0	1042,6	11,86	0	0,04

continua na próxima página

continuação da Tabela 10

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
	GRASP	10,9	959,6	2,95	1440	48,81
	GRASP-R	10,7	961,6	3,17	1412	47,98
	Tabu	11,0	979,0	5,03	1366	7,34
R109	I1	14,0	1457,8	27,11	0	0,00
	VND	14,0	1266,0	10,38	0	0,02
	GRASP	13,0	1166,9	1,74	1418	38,49
	GRASP-R	13,2	1170,2	2,03	1455	38,92
	Tabu	14,0	1229,8	7,23	970	4,52
R110	I1	12,0	1339,0	25,38	0	0,00
	VND	12,0	1193,4	11,74	0	0,01
	GRASP	12,1	1107,5	3,69	1456	42,66
	GRASP-R	12,1	1106,8	3,63	1458	42,71
	Tabu	12,0	1122,4	5,09	1105	5,76
R111	I1	12,0	1276,6	21,73	0	0,00
	VND	12,0	1190,9	13,56	0	0,01
	GRASP	12,0	1070,6	2,09	1424	46,98
	GRASP-R	11,9	1075,1	2,52	1381	44,39
	Tabu	12,0	1066,1	1,66	895	4,65
R112	I1	11,0	1188,1	25,25	0	0,00
	VND	11,0	1072,7	13,08	0	0,02
	GRASP	10,7	982,0	3,52	1376	39,45
	GRASP-R	10,7	983,3	3,65	1574	44,61
	Tabu	11,0	978,2	3,12	1934	10,25
R201	I1	5,0	1805,3	57,92	0	0,00
	VND	5,0	1277,3	11,73	0	0,05
	GRASP	5,1	1195,8	4,61	1468	70,54
	GRASP-R	5,2	1196,0	4,62	1451	70,24
	Tabu	5,0	1216,9	6,45	1371	9,70
R202	I1	4,0	1585,0	53,94	0	0,00
	VND	4,0	1173,8	14,01	0	0,07
	GRASP	5,0	1047,4	1,73	1383	96,12
	GRASP-R	5,1	1049,2	1,90	1348	95,51
	Tabu	4,0	1116,1	8,40	849	6,57
R203	I1	4,0	1534,3	76,19	0	0,00
	VND	4,0	967,4	11,09	0	0,09
	GRASP	4,0	901,7	3,55	1396	128,89
	GRASP-R	4,2	899,2	3,26	1527	142,57
	Tabu	4,0	942,7	8,26	954	7,74

continua na próxima página

continuação da Tabela 10

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
R204	I1	4,0	1155,2	57,97	0	0,00
	VND	4,0	828,9	13,35	0	0,07
	GRASP	3,5	748,7	2,39	1549	144,43
	GRASP-R	3,6	749,3	2,46	1331	123,36
	Tabu	4,0	758,0	3,65	1018	8,34
R205	I1	4,0	1403,3	47,75	0	0,00
	VND	4,0	1117,5	17,66	0	0,05
	GRASP	4,0	979,4	3,11	1295	70,77
	GRASP-R	4,0	980,7	3,25	1398	76,46
	Tabu	4,0	1028,4	8,28	1393	10,77
R206	I1	3,0	1303,5	48,82	0	0,00
	VND	3,0	1034,6	18,12	0	0,06
	GRASP	4,0	902,9	3,08	1277	89,56
	GRASP-R	4,0	901,2	2,89	1521	106,40
	Tabu	3,0	934,0	6,63	1719	14,84
R207	I1	3,0	1169,9	47,34	0	0,00
	VND	3,0	873,5	10,01	0	0,08
	GRASP	3,1	824,6	3,86	1528	135,01
	GRASP-R	3,3	822,6	3,60	1324	118,42
	Tabu	3,0	856,4	7,86	1070	9,48
R208	I1	3,0	974,0	38,94	0	0,00
	VND	3,0	792,7	13,08	0	0,08
	GRASP	3,0	709,8	1,25	1391	120,46
	GRASP-R	3,0	709,5	1,21	1358	119,15
	Tabu	3,0	729,7	4,09	1900	18,38
R209	I1	4,0	1322,5	54,72	0	0,00
	VND	4,0	954,6	11,68	0	0,06
	GRASP	4,0	881,1	3,07	1554	93,79
	GRASP-R	4,0	881,9	3,17	1360	85,29
	Tabu	4,0	913,9	6,91	1563	12,14
R210	I1	4,0	1404,0	55,91	0	0,00
	VND	4,0	973,1	8,06	0	0,07
	GRASP	4,0	931,3	3,42	1497	104,13
	GRASP-R	4,2	931,0	3,39	1356	96,17
	Tabu	4,0	922,1	2,40	1068	8,36
R211	I1	3,0	1013,4	35,72	0	0,00
	VND	3,0	965,4	29,29	0	0,03
	GRASP	3,0	785,2	5,15	1470	84,94

continua na próxima página

continuação da Tabela 10

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
	GRASP-R	3,1	785,4	5,19	1421	84,53
	Tabu	3,0	812,2	8,77	1384	11,69
RC101	I1	17,0	1920,4	18,56	0	0,00
	VND	17,0	1706,6	5,36	0	0,01
	GRASP	16,5	1645,3	1,58	1378	32,75
	GRASP-R	16,7	1646,9	1,67	1437	33,78
	Tabu	17,0	1676,2	3,48	841	3,99
RC102	I1	15,0	1813,3	24,42	0	0,00
	VND	15,0	1533,1	5,19	0	0,03
	GRASP	14,7	1481,1	1,62	1512	47,47
	GRASP-R	14,9	1478,8	1,47	1366	43,16
	Tabu	15,0	1492,6	2,42	1279	6,49
RC103	I1	12,0	1544,5	22,77	0	0,00
	VND	12,0	1334,9	6,11	0	0,03
	GRASP	12,0	1288,5	2,43	1535	53,49
	GRASP-R	11,7	1286,5	2,26	1479	51,48
	Tabu	12,0	1335,3	6,14	839	4,35
RC104	I1	12,0	1380,6	21,93	0	0,00
	VND	12,0	1201,5	6,11	0	0,03
	GRASP	10,7	1163,0	2,71	1379	44,40
	GRASP-R	10,6	1158,5	2,31	1570	49,66
	Tabu	12,0	1197,0	5,71	955	5,13
RC105	I1	16,0	1838,0	21,42	0	0,00
	VND	15,0	1568,1	3,59	0	0,03
	GRASP	15,9	1547,6	2,24	1584	51,94
	GRASP-R	15,3	1540,1	1,74	1450	46,40
	Tabu	15,0	1558,8	2,98	839	4,17
RC106	I1	14,0	1726,5	25,77	0	0,00
	VND	14,0	1542,6	12,38	0	0,02
	GRASP	13,6	1405,5	2,39	1367	36,77
	GRASP-R	13,4	1409,3	2,66	1409	37,30
	Tabu	13,0	1397,7	1,82	1664	8,53
RC107	I1	13,0	1587,4	31,43	0	0,00
	VND	13,0	1296,0	7,30	0	0,02
	GRASP	12,6	1246,0	3,16	1259	38,07
	GRASP-R	12,3	1246,9	3,24	1410	41,50
	Tabu	13,0	1264,4	4,69	954	4,99
	I1	11,0	1368,2	22,80	0	0,00

continua na próxima página

RC108

continuação da Tabela 10

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
	VND	11,0	1183,8	6,25	0	0,03
	GRASP	11,1	1139,6	2,28	1368	40,96
	GRASP-R	11,0	1134,6	1,83	1337	39,43
	Tabu	11,0	1148,9	3,11	923	4,90
RC201	I1	5,0	2025,4	60,52	0	0,00
	VND	5,0	1494,7	18,46	0	0,05
	GRASP	5,4	1316,3	4,32	1323	55,11
	GRASP-R	5,7	1314,5	4,18	1458	61,50
	Tabu	5,0	1350,1	7,00	2240	15,70
RC202	I1	4,0	1783,8	63,31	0	0,00
	VND	4,0	1288,8	17,99	0	0,05
	GRASP	5,0	1116,8	2,24	1191	74,82
	GRASP-R	5,0	1117,9	2,34	1321	83,55
	Tabu	4,0	1244,4	13,93	873	6,76
RC203	I1	4,0	1528,8	65,51	0	0,00
	VND	4,0	1006,8	9,00	0	0,09
	GRASP	4,4	960,1	3,94	1560	119,97
	GRASP-R	4,7	955,4	3,44	1199	90,74
	Tabu	4,0	992,7	7,47	1261	9,79
RC204	I1	4,0	1254,4	60,10	0	0,00
	VND	4,0	837,1	6,84	0	0,06
	GRASP	4,0	790,6	0,91	1429	118,32
	GRASP-R	4,0	791,1	0,97	1332	109,22
	Tabu	4,0	818,2	4,43	1117	8,72
RC205	I1	5,0	2179,4	88,86	0	0,00
	VND	5,0	1380,9	19,66	0	0,06
	GRASP	6,0	1204,1	4,34	1294	71,46
	GRASP-R	6,2	1202,0	4,16	1553	86,04
	Tabu	5,0	1282,2	11,11	1080	7,58
RC206	I1	4,0	1743,1	65,84	0	0,00
	VND	4,0	1092,8	3,97	0	0,06
	GRASP	4,2	1082,2	2,96	1396	71,77
	GRASP-R	4,1	1079,1	2,66	1263	66,10
	Tabu	4,0	1078,6	2,62	974	7,31
RC207	I1	4,0	1565,7	62,60	0	0,00
	VND	4,0	1104,5	14,71	0	0,08
	GRASP	4,9	1002,5	4,11	1348	87,48
	GRASP-R	5,0	1003,5	4,21	1515	99,43

continua na próxima página

continuação da Tabela 10

Instância	Alg.	Veíc.	Custo	Gap (%)	Iter.	Tempo (s)
	Tabu	4,0	1042,1	8,22	871	6,83
	I1	3,0	1155,9	48,94	0	0,00
	VND	3,0	926,6	19,39	0	0,04
RC208	GRASP	3,9	835,2	7,61	1405	73,28
	GRASP-R	4,0	816,3	5,18	1374	71,01
	Tabu	3,0	877,5	13,06	967	8,29

B Invariância das conclusões ao orçamento de parada

O valor de K é um orçamento declarado (Seção 3.13), e não um parâmetro derivado dos dados. Cabe portanto verificar se alguma conclusão do estudo depende dele. A Tabela 11 repete, para cada orçamento medido, o ordenamento dos três métodos iterativos e os testes par a par de Wilcoxon sobre as instâncias de treino. Numa faixa de $64 \times$ (K de 50 a 3200) nem o ordenamento nem o resultado dos testes se alteram.

Tabela 11 – Invariância das conclusões ao orçamento, nas instâncias de treino. Em toda a faixa medida — uma variação de $64 \times$ — o ordenamento dos métodos e o resultado dos testes par a par (Wilcoxon pareado) não se alteram. Nenhuma conclusão do estudo depende do valor de K adotado. Semente única: o *não significativo* entre GRASP e Reativo deve ser lido como ausência de evidência, não como equivalência estabelecida.

K	ordenamento	p (GRASP $\times$ Reativo)	p (GRASP $\times$ Tabu)
50	Reativo < GRASP < Tabu	0,2675	0,0029
100	Reativo < GRASP < Tabu	0,1531	0,0015
200	Reativo < GRASP < Tabu	0,9544	0,0003
400	Reativo < GRASP < Tabu	0,9181	0,0006
800	Reativo < GRASP < Tabu	0,7060	0,0009
1600	Reativo < GRASP < Tabu	0,1591	0,0012
3200	Reativo < GRASP < Tabu	0,7481	0,0005